1 从零开始实现一个Qwen3(Dense)
1.1 简介
1.1.1 Qwen3架构
1.1.2 参考
1.2 结构
1.2.1 RMSNorm（Root Mean Square Layer Normalization）
1.2.2 Rotary Position Embedding (RoPE)
1.2.3 Grouped Query Attention (GQA)
1.2.4 FFN层
1.2.5 Transformer Block
1.2.6 Qwen3-0.6B 完整模型
1.3 文本生成+推理
1.3.1 文本生成
1.3.2 Tokenizer实现
1.3.3 内存优化
1.4 整体代码
2 从零开始实现一个Qwen3(MoE)
2.1 简介
2.1.1 Qwen3 MoE架构
2.1.2 参考
2.2 结构
2.2.1 RMSNorm（Root Mean Square Layer Normalization）
2.2.2 Rotary Position Embedding (RoPE)
2.2.3 Grouped Query Attention (GQA)
2.2.4 MoE前馈网络
2.2.5 标准前馈网络
2.2.6 Transformer Block
2.2.7 Qwen3-30B-A3B 完整模型
2.3 文本生成+推理
2.3.1 文本生成
2.3.2 Tokenizer实现
2.3.3 权重加载
2.4 整体代码

1 从零开始实现一个Qwen3(Dense)

1.1 简介

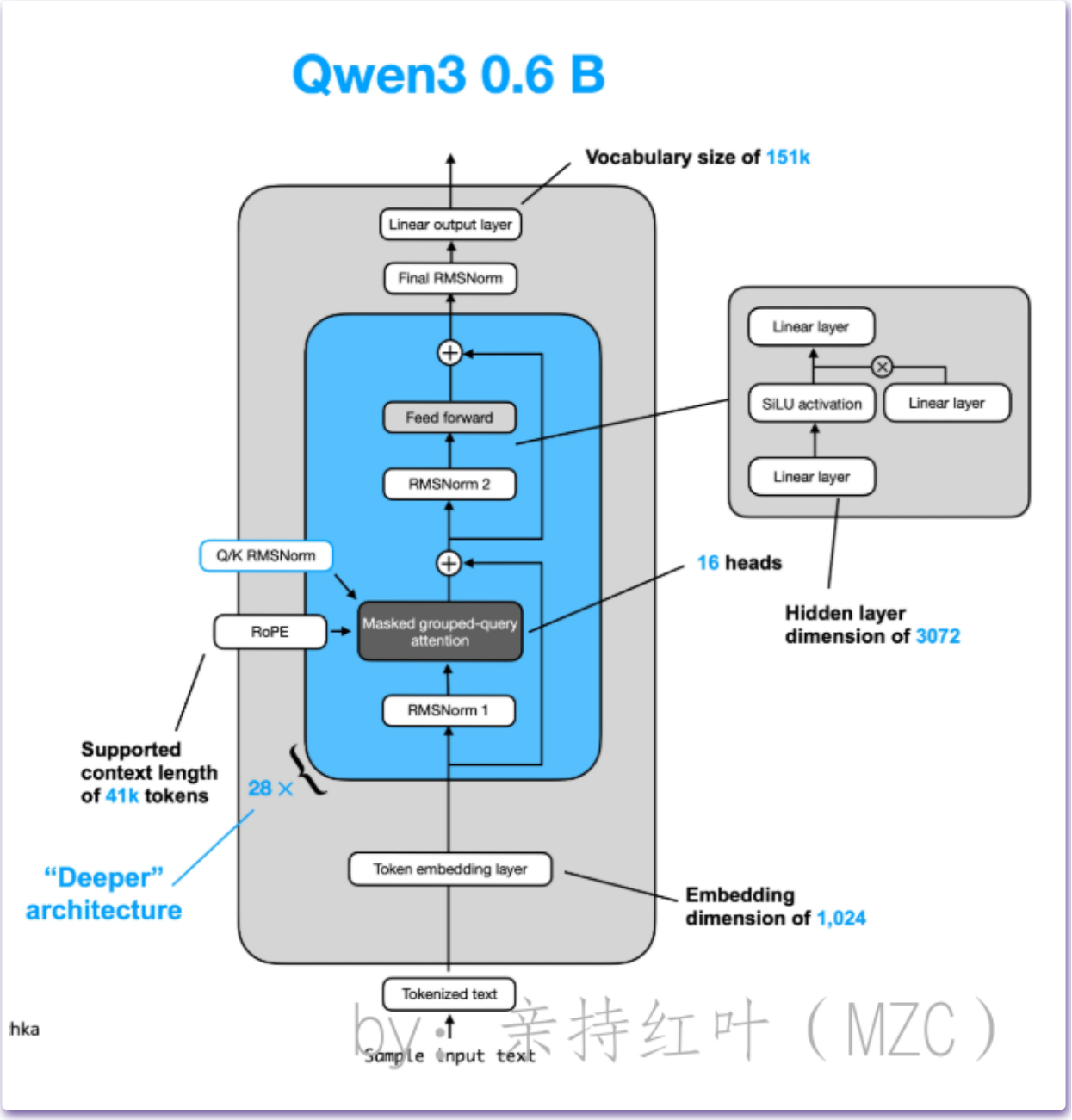
- 实现一个Qwen3-Dense架构，重点讲一下Dencoder部分的结构，剩下的文本生成和推理就一笔带过

1.1.1 Qwen3架构

- 我们结合 **Qwen3-0.6B的config文件**

```
{
  "architectures": [
    "Qwen3ForCausalLM"
  ],
  "attention_bias": false,
  "attention_dropout": 0.0,
  "bos_token_id": 151643,
  "eos_token_id": 151645,
  "head_dim": 128,
  "hidden_act": "silu",
  "hidden_size": 1024,
  "initializer_range": 0.02,
  "intermediate_size": 3072,
  "max_position_embeddings": 40960,
  "max_window_layers": 28,
  "model_type": "qwen3",
  "num_attention_heads": 16,
  "num_hidden_layers": 28,
  "num_key_value_heads": 8,
  "rms_norm_eps": 1e-06,
  "rope_scaling": null,
  "rope_theta": 1000000,
  "sliding_window": null,
  "tie_word_embeddings": true,
  "torch_dtype": "bfloat16",
  "transformers_version": "4.51.0",
  "use_cache": true,
  "use_sliding_window": false,
  "vocab_size": 1
```

以及下面的图



先来简单的看一下Qwen3系列的整体架构（其他dense模型仅仅是参数不同，但是整体架构一致）

1. 词汇表大小： 151936
2. 训练的上下文长度： 40960
3. FFN层的激活函数：SwiGLU （Swish/SiLU + GLU）
4. FFN层隐藏层维度：3072
5. 使用ROPE边骂
6. 使用Pre-Norm，且Normalization函数为RMSNorm
7. 使用Q/K Norm，即在RoPE之前对Q和K使用RMSNorm
8. 使用GQA, q共16个头，G=8

其他的没啥好说的了，使用pydantic定义一个config的数据模型，后续代码中出现的cfg即是这个数据模型的实例化

```python
import torch
from pydantic import BaseModel, Field

class Qwen3Config(BaseModel):
    """
    Pydantic model for Qwen3 configuration
    """
    model_config = {"arbitrary_types_allowed": True}

    vocab_size: int = Field(..., description="词汇表大小")
    context_length: int = Field(..., description="用于训练模型的上下文长度")
    emb_dim: int = Field(..., description="Embedding 维度")
    n_heads: int = Field(..., description="注意力头的数量")
    n_layers: int = Field(..., description="层数")
    hidden_dim: int = Field(..., description="FeedForward 层中中间维度的大小（隐藏层大小）")
    head_dim: int = Field(..., description="GQA 中每个注意力头的维度大小")
    qk_norm: bool = Field(..., description="是否在 GQA 中对key和value进行归一化")
    n_kv_groups: int = Field(..., description="用于分组查询注意力的 KV 组数")
    rope_base: float = Field(..., description="RoPE 中‘theta’的基数值")
    dtype: torch.dtype = Field(..., description="较低精度的数据类型，用于降低显存占用")


QWEN3_CONFIG = {
            "vocab_size": 151_936,  # Vocabulary size
            "context_length": 40_960,  # Context length that was used to train the model
            "emb_dim": 1024,  # Embedding dimension
            "n_heads": 16,  # Number of attention heads
            "n_layers": 28,  # Number of layers
            "hidden_dim": 3072,  # Size of the intermediate dimension in FeedForward
            "head_dim": 128,  # Size of the heads in GQA
            "qk_norm": True,  # Whether to normalize queries and values in GQA
            "n_kv_groups": 8,  # Key-Value groups for grouped-query attention
            "rope_base": 1_000_000.0,  # The base in RoPE's "theta"
            "dtype": torch.bfloat16,  # Lower-precision dtype to reduce memory usage
        }

cfg = Qwen3Config(**QWEN3_CONFIG)
```

1.1.2 参考

- **参考代码**：

> https://github.com/rasbt/LLMs-from-scratch/blob/main/ch05/11_qwen3/standalone-qwen3.ipynb

- **Qwen3: Think Deeper, Act Faster**

> https://qwenlm.github.io/blog/qwen3/

- **Qwen3 Technical Report Qwen3 技术报告**

> https://arxiv.org/abs/2505.09388

1.2 结构

1.2.1 RMSNorm（Root Mean Square Layer Normalization）

1.2.1.1 RMSNorm介绍

- **transformer-RMSNorm**
- RMSNorm通过对神经元输出的均方根值进行缩放，在保持训练稳定性的同时降低计算复杂度。与LayerNorm相比，RMSNorm移除了均值中心化操作，仅通过均方根值进行缩放，减少了约30%的计算量。

  在Qwen3中，RMSNorm被广泛应用于每个Transformer层的归一化操作，包括注意力层前后和前馈网络前后的归一化。该方法通过单一缩放参数调整特征幅度，避免对输入进行平移操作，特别适合大规模模型训练。

1.2.1.2 数学公式

- RMSNorm对输入张量最后一个维度进行均方根标准化：

$$
\begin{aligned}
\mathrm{RMS}(x) &= \sqrt{\frac{1}{emb_dim}\sum_{i=1}^{emb_dim} x_i^2} \\
\mathrm{out} &= \gamma \cdot \frac{x}{\sqrt{\mathrm{RMS}(x)^2+\epsilon}} + \beta
\end{aligned}
\tag{1}
$$

  其中：

  - $\gamma \in \mathbb{R}^{emb_dim}$：可学习缩放参数（初始化为1）
  - β：科学系平移参数（初始化为0）

- ϵ：数值稳定系数（默认1e-6）

emb_dim 为特征维度。

1.2.1.3 代码实现

- Qwen3中的RMSNorm实现

```python
class RMSNorm(nn.Module):
    def __init__(self, emb_dim, eps=1e-6, bias=False, qwen3_compatible=False):
        super().__init__()
        self.eps = eps
        self.qwen3_compatible = qwen3_compatible
        self.scale = nn.Parameter(torch.ones(emb_dim)) # gamma
        self.shift = nn.Parameter(torch.zeros(emb_dim)) if bias else None # beta

    def forward(self, x):
        input_dtype = x.dtype

        if self.qwen3_compatible:
            x = x.to(torch.float32)

        variance = x.pow(2).mean(-1, keepdim=True)
        norm_x = x * torch.rsqrt(variance + self.eps)
        norm_x = norm_x * self.scale

        if self.shift is not None:
            norm_x = norm_x + self.shift

        return norm_x.to(input_dtype)
```

1.2.2 Rotary Position Embedding (RoPE)

1.2.2.1 RoPE介绍

- **transformer-旋转位置编码RoPE**
- 旋转位置编码（Rotary Position Embedding, RoPE）是Qwen3模型中使用的位置编码方法，通过对查询和键向量进行旋转变换来注入位置信息。与传统的绝对位置编码不同，RoPE能够更好地处理相对位置关系，并且对序列长度具有更好的外推能力。

  在Qwen3中，RoPE被应用于每个注意力头的查询和键向量，通过预计算的正弦和余弦值对向量进行旋转变换，使得模型能够感知token之间的相对位置关系。

1.2.2.2 RoPE的数学公式

- RoPE通过复数域的旋转操作实现位置编码：

  对于位置为m的向量$\mathbf{x}$，RoPE变换定义为：

$$f(\mathbf{x}, m) = \mathbf{R}_{\Theta,m}\mathbf{x} \tag{2}$$

其中旋转矩阵$\mathbf{R}_{\Theta,m}$的计算公式为：

$$\mathbf{R}_{\Theta,m} = \begin{pmatrix} \cos(m\theta_0) & -\sin(m\theta_0) & 0 & 0 & \cdots \\ \sin(m\theta_0) & \cos(m\theta_0) & 0 & 0 & \cdots \\ 0 & 0 & \cos(m\theta_1) & -\sin(m\theta_1) & \cdots \\ 0 & 0 & \sin(m\theta_1) & \cos(m\theta_1) & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix} \tag{3}$$

其中$\theta_i = \text{base}^{-2i/d}$，在Qwen3中base=1,000,000。

1.2.2.3 代码实现

- RoPE参数计算函数

```python
def compute_rope_params(head_dim, theta_base=10_000, context_length=4096, dtype=torch.float32):
    assert head_dim % 2 == 0, "Embedding dimension must be even"

    # 计算逆频率
    inv_freq = 1.0 / (theta_base ** (torch.arange(0, head_dim, 2, dtype=dtype)[: (head_dim // 2)].float() / head_dim))

    # 生成位置索引
    positions = torch.arange(context_length, dtype=dtype)

    # 计算角度
    angles = positions[:, None] * inv_freq[None, :]  # Shape: (context_length, head_dim // 2)

    # 扩展角度以匹配head_dim
    angles = torch.cat([angles, angles], dim=1)  # Shape: (context_length, head_dim)

    # 预计算正弦和余弦
    cos = torch.cos(angles)
    sin = torch.sin(angles)

    return cos, sin
```

- RoPE应用函数

```python
def apply_rope(x, cos, sin):
    # x: (batch_size, num_heads, seq_len, head_dim)
    batch_size, num_heads, seq_len, head_dim = x.shape
    assert head_dim % 2 == 0, "Head dimension must be even"

    # 将x分为前半部分和后半部分
    x1 = x[..., : head_dim // 2]  # 前半部分
    x2 = x[..., head_dim // 2:]  # 后半部分

    # 调整sin和cos的形状
```

```
    cos = cos[:seq_len, :].unsqueeze(0).unsqueeze(0)  # Shape: (1, 1, seq_len, head_dim)
    sin = sin[:seq_len, :].unsqueeze(0).unsqueeze(0)

    # 应用旋转变换
    rotated = torch.cat((-x2, x1), dim=-1)
    x_rotated = (x * cos) + (rotated * sin)

    return x_rotated.to(dtype=x.dtype)
```

1.2.3 Grouped Query Attention (GQA)

1.2.3.1 GQA介绍

- **MHA、MQA、GQA：大模型注意力机制的演进**
- 分组查询注意力（Grouped Query Attention, GQA）是Qwen3模型中使用的高效注意力机制。GQA通过减少键值对（Key-Value）的数量来降低内存使用和计算复杂度，同时保持与多头注意力相近的性能。

  在传统的多头注意力中，每个头都有独立的查询、键、值投影。而在GQA中，多个查询头共享同一组键值对，通过分组的方式实现计算效率的提升。Qwen3-0.6B使用8个KV组来支持16个注意力头。

1.2.3.2 数学公式

- GQA的计算过程可以表示为：

$$\mathrm{GQA}(X) = \mathrm{Concat}(\mathrm{head}_1, \mathrm{head}_2, \ldots, \mathrm{head}_h)W^O \tag{4}$$

  其中每个头的计算为：

$$\mathrm{head}_i = \mathrm{Attention}(XW_i^Q, XW_{\lfloor i/g \rfloor}^K, XW_{\lfloor i/g \rfloor}^V) \tag{5}$$

  关键参数：

  - h：查询头的总数
  - g：每组的大小（group_size = num_heads / num_kv_groups）
  - $\lfloor i/g \rfloor$：第i个查询头对应的KV组索引

1.2.3.3 代码实现

- Qwen3中的GQA实现

```
class GroupQueryAttention(nn.Module):
    def __init__(self, d_in, num_heads, num_kv_groups, head_dim=None, qk_norm=False, dtype=None):
        super().__init__()
        assert num_heads % num_kv_groups == 0, "num_heads must be divisible by num_kv_groups"

```

```python
        self.num_heads = num_heads
        self.num_kv_groups = num_kv_groups
        self.group_size = num_heads // num_kv_groups

        if head_dim is None:
            assert d_in % num_heads == 0, "d_in must be divisible by num_heads if head_dim is not set"
            head_dim = d_in // num_heads

        self.head_dim = head_dim
        self.d_out = num_heads * head_dim

        # 查询投影：每个头都有独立的查询投影
        self.W_query = nn.Linear(d_in, self.d_out, bias=False, dtype=dtype)
        # 键值投影：多个头共享键值投影
        self.W_key = nn.Linear(d_in, num_kv_groups * head_dim, bias=False, dtype=dtype)
        self.W_value = nn.Linear(d_in, num_kv_groups * head_dim, bias=False, dtype=dtype)

        self.out_proj = nn.Linear(self.d_out, d_in, bias=False, dtype=dtype)

        # 可选的QK归一化
        if qk_norm:
            self.q_norm = RMSNorm(head_dim, eps=1e-6)
            self.k_norm = RMSNorm(head_dim, eps=1e-6)
        else:
            self.q_norm = self.k_norm = None

    def forward(self, x, mask, cos, sin):
        b, num_tokens, _ = x.shape

        # 应用投影
        queries = self.W_query(x)  # [b, num_tokens, num_heads * head_dim]
        keys = self.W_key(x)       # [b, num_tokens, num_kv_groups * head_dim]
        values = self.W_value(x)   # [b, num_tokens, num_kv_groups * head_dim]

        # 重塑张量
        queries = queries.view(b, num_tokens, self.num_heads, self.head_dim).transpose(1, 2)
        keys = keys.view(b, num_tokens, self.num_kv_groups, self.head_dim).transpose(1, 2)
        values = values.view(b, num_tokens, self.num_kv_groups, self.head_dim).transpose(1, 2)

        # 可选的QK归一化
        if self.q_norm:
```

```python
            queries = self.q_norm(queries)
        if self.k_norm:
            keys = self.k_norm(keys)

        # 应用RoPE位置编码
        queries = apply_rope(queries, cos, sin)
        keys = apply_rope(keys, cos, sin)

        # 扩展K和V以匹配查询头的数量
        keys = keys.repeat_interleave(self.group_size, dim=1)
        values = values.repeat_interleave(self.group_size, dim=1)

        # 计算注意力
        attn_scores = queries @ keys.transpose(2, 3)
        attn_scores = attn_scores.masked_fill(mask, -torch.inf)
        attn_weights = torch.softmax(attn_scores / self.head_dim ** 0.5, dim=-1)

        context = (attn_weights @ values).transpose(1, 2).reshape(b, num_tokens, self.d_out)
        return self.out_proj(context)
```

- 关键特性解析

特性	说明	Qwen3-0.6B配置
num_heads	查询头数量	16
num_kv_groups	键值组数量	8
group_size	每组大小	2
head_dim	每个头的维度	128
qk_norm	QK归一化	True

- 维度变化流程

操作步骤	张量形状变化示例
输入数据	[batch_size, seq_len, emb_dim]
查询投影	[batch_size, seq_len, num_heads * head_dim]
键值投影	[batch_size, seq_len, num_kv_groups * head_dim]
重塑查询	[batch_size, num_heads, seq_len, head_dim]
重塑键值	[batch_size, num_kv_groups, seq_len, head_dim]
扩展键值	[batch_size, num_heads, seq_len, head_dim]
注意力计算	[batch_size, num_heads, seq_len, head_dim]
输出投影	[batch_size, seq_len, emb_dim]

1.2.4 FFN层

1.2.4.1 SwiGLU介绍

- **GLU 变种：ReGLU 、 GEGLU 、 SwiGLU**
- SwiGLU（Swish-Gated Linear Unit）是Qwen3模型中使用的激活函数，结合了Swish激活函数和门控机制。SwiGLU通过两个并行的线性变换和门控机制，提供了比传统ReLU更好的表达能力和梯度流动特性。

  在Qwen3的前馈网络中，SwiGLU替代了传统的ReLU激活函数，通过门控机制选择性地激活神经元，提高了模型的非线性建模能力。该激活函数在大规模语言模型中表现出色，能够更好地处理复杂的语言模式。

1.2.4.2 数学公式

- SwiGLU的计算公式为：

$$\mathrm{SwiGLU}(x) = \mathrm{Swish}(xW_1 + b_1) \odot (xW_2 + b_2) \tag{6}$$

  其中：
  - $\mathrm{Swish}(x) = x \cdot \sigma(\beta x) = x \cdot \mathrm{sigmoid}(\beta x)$
  - 当$\beta = 1$时，$\mathrm{Swish}(x) = x \cdot \mathrm{sigmoid}(x) = \mathrm{SiLU}(x)$
  - $\odot$ 表示逐元素乘法（Hadamard积）
  - W_1, W_2 是两个独立的权重矩阵
  - b_1, b_2 是偏置项（**在Qwen3中通常省略**）

  完整的前馈网络公式：

$$\mathrm{FFN}(x) = \mathrm{SwiGLU}(x)W_3 = (\mathrm{SiLU}(xW_1) \odot xW_2)W_3 \tag{7}$$

1.2.4.3 代码实现

- Qwen3中的SwiGLU前馈网络实现

```python
class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        # 两个并行的线性层用于门控机制
        self.fc1 = nn.Linear(cfg.emb_dim, cfg.hidden_dim, dtype=cfg.dtype, bias=False)
        self.fc2 = nn.Linear(cfg.emb_dim, cfg.hidden_dim, dtype=cfg.dtype, bias=False)
        # 输出投影层
        self.fc3 = nn.Linear(cfg.hidden_dim, cfg.emb_dim, dtype=cfg.dtype, bias=False)

    def forward(self, x):
        """SwiGLU激活函数"""
        # 第一个分支：应用SiLU激活
        x_fc1 = self.fc1(x)
        # 第二个分支：线性变换
        x_fc2 = self.fc2(x)
        # 门控机制：SiLU(x_fc1) * x_fc2
        x = F.silu(x_fc1) * x_fc2
        # 输出投影
        return self.fc3(x)
```

- 相关解释
    1. `网络结构特点`：
        - **双分支设计**: fc1和fc2形成两个并行分支
        - **门控机制**: 通过逐元素乘法实现选择性激活
        - **无偏置**: 所有线性层都不使用偏置项，简化计算
    2. `激活函数选择`：
        - **SiLU**: $\mathrm{SiLU}(x) = x \cdot \mathrm{sigmoid}(x)$
        - **平滑性**: 相比ReLU，SiLU在整个定义域内可微
        - **门控效应**: 通过sigmoid函数实现软门控
- 维度变化流程

操作步骤	张量形状变化示例	Qwen3-0.6B配置
输入数据	[batch_size, seq_len, emb_dim]	[4, 100, 1024]
fc1分支	[batch_size, seq_len, hidden_dim]	[4, 100, 3072]
fc2分支	[batch_size, seq_len, hidden_dim]	[4, 100, 3072]
SiLU激活	[batch_size, seq_len, hidden_dim]	[4, 100, 3072]
门控乘法	[batch_size, seq_len, hidden_dim]	[4, 100, 3072]
fc3输出	[batch_size, seq_len, emb_dim]	[4, 100, 1024]

1.2.5 Transformer Block

- 后续注意，实际现在llm基本都是Decoder-only，所以后面的Transformer Block一律默认为Decoder Block

1.2.5.1 Transformer Block介绍

- Qwen3的Transformer Block是模型的核心构建单元，每个Block包含一个分组查询注意力层和一个SwiGLU前馈网络层。与标准Transformer不同，Qwen3采用了Pre-Norm结构，即在每个子层之前应用RMSNorm，这种设计有助于训练稳定性和梯度流动。

  每个Transformer Block通过残差连接将输入与子层输出相加，形成深度网络中的信息高速公路。Qwen3-0.6B模型包含28个这样的Transformer Block，通过层层堆叠实现复杂的语言理解和生成能力。

1.2.5.2 数学表达

- Transformer Block的计算过程可以表示为：

  注意力子层：

$$\text{Attn_Out} = x + \text{GQA}(\text{RMSNorm}(x), \text{mask}, \cos, \sin) \tag{8}$$

  前馈子层：

$$\text{Block_Out} = \text{Attn_Out} + \text{SwiGLU}(\text{RMSNorm}(\text{Attn_Out})) \tag{9}$$

  其中：

  - x 是输入张量
  - GQA 是分组查询注意力
  - SwiGLU 是使用SwiGLU的FFN层
  - RMSNorm 是均方根归一化
  - $\cos, \sin$ 是RoPE位置编码参数

1.2.5.3 代码实现

- Qwen3中的Transformer Block实现

```python
class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        # 分组查询注意力层
        self.att = GroupQueryAttention(
            d_in=cfg.emb_dim,
            num_heads=cfg.n_heads,
            head_dim=cfg.head_dim,
            num_kv_groups=cfg.n_kv_groups,
            qk_norm=cfg.qk_norm,
            dtype=cfg.dtype
        )
        # SwiGLU前馈网络
        self.ff = FeedForward(cfg)
        # 两个RMSNorm层
        self.norm1 = RMSNorm(cfg.emb_dim, eps=1e-6)
        self.norm2 = RMSNorm(cfg.emb_dim, eps=1e-6)

    def forward(self, x, mask, cos, sin):
        # 注意力子层（Pre-Norm结构）
        shortcut = x
        x = self.norm1(x)
        x = self.att(x, mask, cos, sin)
        x = x + shortcut  # 残差连接

        # 前馈子层（Pre-Norm结构）
        shortcut = x
        x = self.norm2(x)
        x = self.ff(x)
        x = x + shortcut  # 残差连接

        return x
```

- 关键设计特点
    1. `Pre-Norm结构`：
        - 在每个子层之前应用归一化，而不是之后
        - 有助于训练稳定性，特别是在深层网络中
        - 减少梯度消失问题
    2. `残差连接`：
        - 每个子层都有直接的跳跃连接
        - 保证信息流动和梯度传播
        - 使得深层网络训练成为可能
- 维度变化流程

操作步骤	张量形状变化示例
输入数据	[batch_size, seq_len, emb_dim]
注意力归一化	[batch_size, seq_len, emb_dim]
分组查询注意力	[batch_size, seq_len, emb_dim]
第一次残差连接	[batch_size, seq_len, emb_dim]
前馈归一化	[batch_size, seq_len, emb_dim]
SwiGLU前馈	[batch_size, seq_len, emb_dim]
第二次残差连接	[batch_size, seq_len, emb_dim]

1.2.6 Qwen3-0.6B 完整模型

1.2.6.1 模型架构

- Qwen3的完整架构包含以下组件：
    1. **词嵌入层（Token Embedding）**：将输入token映射为高维向量
    2. **多层Transformer Block**：核心的特征提取和变换层
    3. **最终归一化层（Final Norm）**：输出前的最后一次归一化
    4. **输出投影层（Output Head）**：将隐藏状态映射到词汇表空间

  模型支持因果语言建模，通过上三角掩码确保生成过程中不会看到未来信息。

1.2.6.2 代码实现

- Qwen3完整模型实现

```
class Qwen3Model(nn.Module):
    def __init__(self, cfg):
        super().__init__()

        # 主要模型参数
        self.tok_emb = nn.Embedding(cfg.vocab_size, cfg.emb_dim, dtype=cfg.dtype)
        self.trf_blocks = nn.ModuleList(
            [TransformerBlock(cfg) for _ in range(cfg.n_layers)]
        )
        self.final_norm = RMSNorm(cfg.emb_dim)
        self.out_head = nn.Linear(cfg.emb_dim, cfg.vocab_size, bias=False, dtype=cfg.dtype)

        # 可重用的工具组件
        if cfg.head_dim is None:
            head_dim = cfg.emb_dim // cfg.n_heads
        else:
```

```
            head_dim = cfg.head_dim

        cos, sin = compute_rope_params(
            head_dim=head_dim,
            theta_base=cfg.rope_base,
            context_length=cfg.context_length
        )
        self.register_buffer("cos", cos, persistent=False)
        self.register_buffer("sin", sin, persistent=False)
        self.cfg = cfg

    def forward(self, in_idx):
        # 前向传播
        tok_embeds = self.tok_emb(in_idx)
        x = tok_embeds

        num_tokens = x.shape[1]
        # 创建因果掩码
        mask = torch.triu(
            torch.ones(
                num_tokens, num_tokens, device=x.device, dtype=torch.bool
            ),
            diagonal=1
        )

        # 通过所有Transformer Block
        for block in self.trf_blocks:
            x = block(x, mask, self.cos, self.sin)

        # 最终归一化和输出投影
        x = self.final_norm(x)
        logits = self.out_head(x.to(self.cfg.dtype))
        return logits
```

- 模型配置系统

```
def set_qwen3_config(choose_model="0.6B"):
    assert choose_model in ["0.6B", "1.7B", "4B", "8B", "14B", "32B"]

    if choose_model == "0.6B":
        QWEN3_CONFIG = {
            "vocab_size": 151_936,
            "context_length": 40_960,
            "emb_dim": 1024,
            "n_heads": 16,
            "n_layers": 28,
            "hidden_dim": 3072,
            "head_dim": 128,
            "qk_norm": True,
            "n_kv_groups": 8,
```

```
            "rope_base": 1_000_000.0,
            "dtype": torch.bfloat16,
        }
    # ... 其他配置这里先不写，后续看整体代码，

    return Qwen3Config(**QWEN3_CONFIG)
```

- 不同规模模型对比

模型规模	参数量	嵌入维度	注意力头	层数	隐藏维度
0.6B	0.6B	1024	16	28	3072
1.7B	1.7B	2048	16	28	6144
4B	4B	2560	32	36	9728
8B	8B	4096	32	36	12288
14B	14B	5120	40	40	17408
32B	32B	5120	64	64	25600

- qwen3-0.6B加载后的模型print

```
model:
Qwen3Model(
  (tok_emb): Embedding(151936, 1024)
  (trf_blocks): ModuleList(
    (0-27): 28 x TransformerBlock(
        (W_query): Linear(in_features=1024, out_features=2048, bias=False)
        (W_key): Linear(in_features=1024, out_features=1024, bias=False)
        (W_value): Linear(in_features=1024, out_features=1024, bias=False)
        (out_proj): Linear(in_features=2048, out_features=1024, bias=False)
        (q_norm): RMSNorm()
        (k_norm): RMSNorm()
      )
      (ff): FeedForward(
        (fc1): Linear(in_features=1024, out_features=3072, bias=False)
        (fc2): Linear(in_features=1024, out_features=3072, bias=False)
        (fc3): Linear(in_features=3072, out_features=1024, bias=False)
      )
      (norm1): RMSNorm()
      (norm2): RMSNorm()
    )
```

```
  )
  (final_norm): RMSNorm()
  (out_head): Linear(in_features=1024, out_features=151936, bias=False)
)
```

1.3 文本生成+推理

1.3.1 文本生成

- 基础的贪婪解码算法：

```python
def generate_text_basic_stream(model, token_ids, max_new_tokens, eos_token_id=None):
    model.eval()
    with torch.no_grad():
        for _ in range(max_new_tokens):
            # 前向传播获取logits
            out = model(token_ids)[:, -1]  # 只取最后一个位置的输出

            # 贪婪解码：选择概率最高的token
            next_token = torch.argmax(out, dim=-1, keepdim=True)

            # 检查是否遇到结束token
            if (eos_token_id is not None and torch.all(next_token == eos_token_id)):
                break

            yield next_token

            # 将新token添加到序列中
            token_ids = torch.cat([token_ids, next_token], dim=1)
```

1.3.2 Tokenizer实现

- Qwen3专用的Tokenizer实现

```python
class Qwen3Tokenizer:
    _SPECIALS = [
        "<|endoftext|>", "<|im_start|>", "<|im_end|>",
        "<|object_ref_start|>", "<|object_ref_end|>",
        "<|box_start|>", "<|box_end|>",
        "<|quad_start|>", "<|quad_end|>",
        "<|vision_start|>", "<|vision_end|>",
        "<|vision_pad|>", "<|image_pad|>", "<|video_pad|>",
    ]
    _SPLIT_RE = re.compile(r"(<\|[^>]+?\|>)")
```

```python

    def __init__(self, tokenizer_file_path="tokenizer.json", repo_id=None,
                 apply_chat_template=True, add_generation_prompt=False, add_thinking=False):
        self.apply_chat_template = apply_chat_template
        self.add_generation_prompt = add_generation_prompt
        self.add_thinking = add_thinking

        tok_file = Path(tokenizer_file_path)
        self._tok = Tokenizer.from_file(str(tok_file))
        self._special_to_id = {t: self._tok.token_to_id(t) for t in self._SPECIALS}

        self.pad_token_id = self._special_to_id.get("<|endoftext|>")
        self.eos_token_id = self.pad_token_id

    def encode(self, text, chat_wrapped=None):
        if chat_wrapped is None:
            chat_wrapped = self.apply_chat_template

        if chat_wrapped:
            text = self._wrap_chat(text)

        ids = []
        for part in filter(None, self._SPLIT_RE.split(text)):
            if part in self._special_to_id:
                ids.append(self._special_to_id[part])
            else:
                ids.extend(self._tok.encode(part).ids)
        return ids

    def decode(self, ids):
        return self._tok.decode(ids, skip_special_tokens=False)

    def _wrap_chat(self, user_msg):
        s = f"<|im_start|>user\n{user_msg}<|im_end|>\n"
        if self.add_generation_prompt:
            s += "<|im_start|>assistant"
            if self.add_thinking:
                s += "\n"
            else:
                s += "\n<think>\n\n</think>\n\n"
        return s
```

1.3.3 内存优化

- Qwen3模型采用了多种内存优化技术：
    1. **混合精度训练**：使用bfloat16减少内存使用

2. **分组查询注意力**：减少KV缓存大小
3. **权重共享**：embedding层和输出层共享权重

```python
def model_memory_size(model, input_dtype=torch.float32):
    total_params = 0
    total_grads = 0
    for param in model.parameters():
        param_size = param.numel()
        total_params += param_size
        if param.requires_grad:
            total_grads += param_size

    total_buffers = sum(buf.numel() for buf in model.buffers())
    element_size = torch.tensor(0, dtype=input_dtype).element_size()
    total_memory_bytes = (total_params + total_grads + total_buffers) * element_size
    total_memory_gb = total_memory_bytes / (1024 ** 3)

    return total_memory_gb
```

- 不同精度下的内存使用对比：

模型规模	float32	bfloat16	内存节省
0.6B	2.4 GB	1.2 GB	50%
1.7B	6.8 GB	3.4 GB	50%
4B	16.0 GB	8.0 GB	50%
8B	32.0 GB	16.0 GB	50%

1.4 整体代码

- 整体代码如下

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-
#
# @File    : 1_qwen3_architecture.py.py
# @Date    : 2025/8/3
# @Author  : mengzhichao
# @Version : 1.0
# @Desc    :

import torch
from torch import nn
from torch.nn import functional as F
```

```python
from pydantic import BaseModel, Field
import json
import os
from pathlib import Path
from safetensors.torch import load_file
from huggingface_hub import hf_hub_download, snapshot_download
import re
from tokenizers import Tokenizer
from tqdm import tqdm
import time

class Qwen3Config(BaseModel):
    """
    Pydantic model for Qwen3 configuration
    """
    model_config = {"arbitrary_types_allowed": True}

    vocab_size: int = Field(..., description="词汇表大小")
    context_length: int = Field(..., description="用于训练模型的上下文长度")
    emb_dim: int = Field(..., description="Embedding 维度")
    n_heads: int = Field(..., description="注意力头的数量")
    n_layers: int = Field(..., description="层数")
    hidden_dim: int = Field(..., description="FeedForward 层中中间维度的大小（隐藏层大小）")
    head_dim: int = Field(..., description="GQA 中每个注意力头的维度大小")
    qk_norm: bool = Field(..., description="是否在 GQA 中对key和value进行归一化")
    n_kv_groups: int = Field(..., description="用于分组查询注意力的 KV 组数")
    rope_base: float = Field(..., description="RoPE 中‘theta’的基数值")
    dtype: torch.dtype = Field(..., description="较低精度的数据类型，用于降低显存占用")


class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.fc1 = nn.Linear(cfg.emb_dim, cfg.hidden_dim, dtype=cfg.dtype, bias=False)
        self.fc2 = nn.Linear(cfg.emb_dim, cfg.hidden_dim, dtype=cfg.dtype, bias=False)
        self.fc3 = nn.Linear(cfg.hidden_dim, cfg.emb_dim, dtype=cfg.dtype, bias=False)

    def forward(self, x):
        """SwiGLU"""
        x_fc1 = self.fc1(x)
        x_fc2 = self.fc2(x)
        x = F.silu(x_fc1) * x_fc2
```

```python
        return self.fc3(x)


class RMSNorm(nn.Module):
    def __init__(self, emb_dim, eps=1e-6, bias=False, qwen3_compatible=False):
        super().__init__()
        self.eps = eps
        self.qwen3_compatible = qwen3_compatible
        self.scale = nn.Parameter(torch.ones(emb_dim)) # gamma
        self.shift = nn.Parameter(torch.zeros(emb_dim)) if bias else None # beta

    def forward(self, x):
        input_dtype = x.dtype

        if self.qwen3_compatible:
            x = x.to(torch.float32)

        variance = x.pow(2).mean(-1, keepdim=True)
        norm_x = x * torch.rsqrt(variance + self.eps)
        norm_x = norm_x * self.scale

        if self.shift is not None:
            norm_x = norm_x + self.shift

        return norm_x.to(input_dtype)


def compute_rope_params(head_dim, theta_base=10_000, context_length=4096, dtype=torch.float32):
    assert head_dim % 2 == 0, "Embedding dimension must be even"

    # Compute the inverse frequencies
    inv_freq = 1.0 / (theta_base ** (torch.arange(0, head_dim, 2, dtype=dtype)[: (head_dim // 2)].float() / head_dim))

    # Generate position indices
    positions = torch.arange(context_length, dtype=dtype)

    # Compute the angles
    angles = positions[:, None] * inv_freq[None, :]  # Shape: (context_length, head_dim // 2)

    # Expand angles to match the head_dim
    angles = torch.cat([angles, angles], dim=1)  # Shape: (context_length, head_dim)

    # Precompute sine and cosine
    cos = torch.cos(angles)
    sin = torch.sin(angles)

```

```python
    return cos, sin


def apply_rope(x, cos, sin):
    # x: (batch_size, num_heads, seq_len, head_dim)
    batch_size, num_heads, seq_len, head_dim = x.shape
    assert head_dim % 2 == 0, "Head dimension must be even"

    # Split x into first half and second half
    x1 = x[..., : head_dim // 2]  # First half
    x2 = x[..., head_dim // 2:]  # Second half

    # Adjust sin and cos shapes
    cos = cos[:seq_len, :].unsqueeze(0).unsqueeze(0)  # Shape: (1, 1, seq_len, head_dim)
    sin = sin[:seq_len, :].unsqueeze(0).unsqueeze(0)

    # Apply the rotary transformation
    rotated = torch.cat((-x2, x1), dim=-1)
    x_rotated = (x * cos) + (rotated * sin)

    # It's ok to use lower-precision after applying cos and sin rotation
    return x_rotated.to(dtype=x.dtype)


class GroupQueryAttention(nn.Module):
    def __init__(self, d_in, num_heads, num_kv_groups, head_dim=None, qk_norm=False, dtype=None):
        super().__init__()
        assert num_heads % num_kv_groups == 0, "num_heads must be divisible by num_kv_groups"

        self.num_heads = num_heads
        self.num_kv_groups = num_kv_groups
        self.group_size = num_heads // num_kv_groups

        if head_dim is None:
            assert d_in % num_heads == 0, "d_in must be divisible by num_heads if head_dim is not set"
            head_dim = d_in // num_heads

        self.head_dim = head_dim
        self.d_out = num_heads * head_dim

        self.W_query = nn.Linear(d_in, self.d_out, bias=False, dtype=dtype)
        self.W_key = nn.Linear(d_in, num_kv_groups * head_dim, bias=False, dtype=dtype)
        self.W_value = nn.Linear(d_in, num_kv_groups * head_dim, bias=False, dtype=dtype)

```

```python
        self.out_proj = nn.Linear(self.d_out, d_in, bias=False, dtype=dtype)

        if qk_norm:
            self.q_norm = RMSNorm(head_dim, eps=1e-6)
            self.k_norm = RMSNorm(head_dim, eps=1e-6)
        else:
            self.q_norm = self.k_norm = None

    def forward(self, x, mask, cos, sin):
        b, num_tokens, _ = x.shape

        # Apply projections
        queries = self.W_query(x)  # [b, num_tokens, num_heads * head_dim]
        keys = self.W_key(x)  # [b, num_tokens, num_kv_groups * head_dim]
        values = self.W_value(x)  # [b, num_tokens, num_kv_groups * head_dim]

        # Reshape
        queries = queries.view(b, num_tokens, self.num_heads, self.head_dim).transpose(1,
                                2)  # [b, num_heads, num_tokens, head_dim]
        keys = keys.view(b, num_tokens, self.num_kv_groups, self.head_dim).transpose(1,
                                2)  # [b, num_kv_groups,head_dim, num_tokens,head_dim]
        values = values.view(b, num_tokens, self.num_kv_groups, self.head_dim).transpose(1,
                                2)  # [b, num_kv_groups,head_dim, num_tokens,head_dim]

        # Optional normalization
        if self.q_norm:
            queries = self.q_norm(queries)
        if self.k_norm:
            keys = self.k_norm(keys)

        # Apply RoPE
        queries = apply_rope(queries, cos, sin)
        keys = apply_rope(keys, cos, sin)

        # Expand K and V to match number of heads
        keys = keys.repeat_interleave(self.group_size, dim=1)
        values = values.repeat_interleave(self.group_size, dim=1)

        # Attention
        attn_scores = queries @ keys.transpose(2, 3)
```

```python
        attn_scores = attn_scores.masked_fill(mask, -torch.inf)
        attn_weights = torch.softmax(attn_scores / self.head_dim ** 0.5, dim=-1)

        context = (attn_weights @ values).transpose(1, 2).reshape(b, num_tokens, self.d_out)
        return self.out_proj(context)


class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.att = GroupQueryAttention(
            d_in=cfg.emb_dim,
            num_heads=cfg.n_heads,
            head_dim=cfg.head_dim,
            num_kv_groups=cfg.n_kv_groups,
            qk_norm=cfg.qk_norm,
            dtype=cfg.dtype
        )
        self.ff = FeedForward(cfg)
        self.norm1 = RMSNorm(cfg.emb_dim, eps=1e-6)
        self.norm2 = RMSNorm(cfg.emb_dim, eps=1e-6)

    def forward(self, x, mask, cos, sin):
        # shortcut connection for attention block
        shortcut = x
        x = self.norm1(x)
        x = self.att(x, mask, cos, sin)  # Shape [batch_size,num_tokens,emb_size]
        x = x + shortcut

        # shortcut connection for feed-forward block
        shortcut = x
        x = self.norm2(x)
        x = self.ff(x)
        x = x + shortcut  # Add the original input back

        return x


class Qwen3Model(nn.Module):
    def __init__(self, cfg):
        super().__init__()

        # Main model parameters
        self.tok_emb = nn.Embedding(cfg.vocab_size, cfg.emb_dim, dtype=cfg.dtype)
        self.trf_blocks = nn.ModuleList(
            # ModuleList since Sequential can only accept one input, and we need `x, mask, cos, sin`
            [TransformerBlock(cfg) for _ in range(cfg.n_layers)]
```

```python
        )
        self.final_norm = RMSNorm(cfg.emb_dim)
        self.out_head = nn.Linear(cfg.emb_dim, cfg.vocab_size, bias=False, dtype=cfg.dtype)

        # Reusuable utilities
        if cfg.head_dim is None:
            head_dim = cfg.emb_dim // cfg.n_heads
        else:
            head_dim = cfg.head_dim

        cos, sin = compute_rope_params(
            head_dim=head_dim,
            theta_base=cfg.rope_base,
            context_length=cfg.context_length
        )
        self.register_buffer("cos", cos, persistent=False)
        self.register_buffer("sin", sin, persistent=False)
        self.cfg = cfg

    def forward(self, in_idx):
        # Forward pass
        tok_embeds = self.tok_emb(in_idx)
        x = tok_embeds

        num_tokens = x.shape[1]
        mask = torch.triu(
            torch.ones(
                num_tokens, num_tokens, device=x.device, dtype=torch.bool
            ),
            diagonal=1
        )

        for block in self.trf_blocks:
            x = block(x, mask, self.cos, self.sin)
        x = self.final_norm(x)
        logits = self.out_head(x.to(self.cfg.dtype))
        return logits


def set_qwen3_config(choose_model="0.6B"):
    assert choose_model in ["0.6B", "1.7B", "4B", "8B", "14B", "32B"], "model choose error, must be one of [0.6B, 1.7B, 4B, 8B, 14B, 32B]"

    CHOOSE_MODEL = choose_model

    if CHOOSE_MODEL == "0.6B":
        QWEN3_CONFIG = {
            "vocab_size": 151_936,  # Vocabulary size
```

```
            "context_length": 40_960,  # Context length that was used to train the model
            "emb_dim": 1024,  # Embedding dimension
            "n_heads": 16,  # Number of attention heads
            "n_layers": 28,  # Number of layers
            "hidden_dim": 3072,  # Size of the intermediate dimension in FeedForward
            "head_dim": 128,  # Size of the heads in GQA
            "qk_norm": True,  # Whether to normalize queries and values in GQA
            "n_kv_groups": 8,  # Key-Value groups for grouped-query attention
            "rope_base": 1_000_000.0,  # The base in RoPE's "theta"
            "dtype": torch.bfloat16,  # Lower-precision dtype to reduce memory usage
        }

    elif CHOOSE_MODEL == "1.7B":
        QWEN3_CONFIG = {
            "vocab_size": 151_936,
            "context_length": 40_960,
            "emb_dim": 2048,  # 2x larger than above
            "n_heads": 16,
            "n_layers": 28,
            "hidden_dim": 6144,  # 2x larger than above
            "head_dim": 128,
            "qk_norm": True,
            "n_kv_groups": 8,
            "rope_base": 1_000_000.0,
            "dtype": torch.bfloat16,
        }

    elif CHOOSE_MODEL == "4B":
        QWEN3_CONFIG = {
            "vocab_size": 151_936,
            "context_length": 40_960,
            "emb_dim": 2560,  # 25% larger than above
            "n_heads": 32,  # 2x larger than above
            "n_layers": 36,  # 29% larger than above
            "hidden_dim": 9728,  # ~3x larger than above
            "head_dim": 128,
            "qk_norm": True,
            "n_kv_groups": 8,
            "rope_base": 1_000_000.0,
            "dtype": torch.bfloat16,
        }

    elif CHOOSE_MODEL == "8B":
        QWEN3_CONFIG = {
            "vocab_size": 151_936,
            "context_length": 40_960,
```

```
            "emb_dim": 4096,  # 60% larger than above
            "n_heads": 32,
            "n_layers": 36,  # 26% larger than above
            "hidden_dim": 12288,
            "head_dim": 128,
            "qk_norm": True,
            "n_kv_groups": 8,
            "rope_base": 1_000_000.0,
            "dtype": torch.bfloat16,
        }

    elif CHOOSE_MODEL == "14B":
        QWEN3_CONFIG = {
            "vocab_size": 151_936,
            "context_length": 40_960,
            "emb_dim": 5120,  # 25% larger than above
            "n_heads": 40,  # 25% larger than above
            "n_layers": 40,  # 11% larger than above
            "hidden_dim": 17408,  # 42% larger than above
            "head_dim": 128,
            "qk_norm": True,
            "n_kv_groups": 8,
            "rope_base": 1_000_000.0,
            "dtype": torch.bfloat16,
        }
    elif CHOOSE_MODEL == "32B":
        QWEN3_CONFIG = {
            "vocab_size": 151_936,
            "context_length": 40_960,
            "emb_dim": 5120,
            "n_heads": 64,  # 60% larger than above
            "n_layers": 64,  # 60% larger than above
            "hidden_dim": 25600,  # 47% larger than above
            "head_dim": 128,
            "qk_norm": True,
            "n_kv_groups": 8,
            "rope_base": 1_000_000.0,
            "dtype": torch.bfloat16,
        }

    else:
        raise ValueError(f"{CHOOSE_MODEL} is not supported.")

    return Qwen3Config(**QWEN3_CONFIG)


def load_weights_into_qwen(model, param_config, params):
    def assign(left, right, tensor_name="unknown"):
        if left.shape != right.shape:
            raise ValueError(f"Shape mismatch in tensor
'{tensor_name}'. Left: {left.shape}, Right: {right.shape}")
```

```python
        return torch.nn.Parameter(right.clone().detach() if isinstance(right, torch.Tensor) else torch.tensor(right))

    model.tok_emb.weight = assign(model.tok_emb.weight, params["model.embed_tokens.weight"],
                                  "model.embed_tokens.weight")

    bar = tqdm(range(param_config.n_layers))
    for l in bar:
        bar.set_description(f"Loading weights into layer {l+1}")
        block = model.trf_blocks[l]
        att = block.att

        # Q, K, V projections
        att.W_query.weight = assign(
            att.W_query.weight,
            params[f"model.layers.{l}.self_attn.q_proj.weight"],
            f"model.layers.{l}.self_attn.q_proj.weight"
        )
        att.W_key.weight = assign(
            att.W_key.weight,
            params[f"model.layers.{l}.self_attn.k_proj.weight"],
            f"model.layers.{l}.self_attn.k_proj.weight"
        )
        att.W_value.weight = assign(
            att.W_value.weight,
            params[f"model.layers.{l}.self_attn.v_proj.weight"],
            f"model.layers.{l}.self_attn.v_proj.weight"
        )

        # Output projection
        att.out_proj.weight = assign(
            att.out_proj.weight,
            params[f"model.layers.{l}.self_attn.o_proj.weight"],
            f"model.layers.{l}.self_attn.o_proj.weight"
        )

        # QK norms
        if hasattr(att, "q_norm") and att.q_norm is not None:
            att.q_norm.scale = assign(
                att.q_norm.scale,
                params[f"model.layers.{l}.self_attn.q_norm.weight"],
                f"model.layers.{l}.self_attn.q_norm.weight"
            )
        if hasattr(att, "k_norm") and att.k_norm is not None:
            att.k_norm.scale = assign(
                att.k_norm.scale,
                params[f"model.layers.{l}.self_attn.k_norm.weight"],
                f"model.layers.{l}.self_attn.k_norm.weight"
            )
```

```python

        # Attention layernorm
        block.norm1.scale = assign(
            block.norm1.scale,
            params[f"model.layers.{l}.input_layernorm.weight"],
            f"model.layers.{l}.input_layernorm.weight"
        )

        # Feedforward weights
        block.ff.fc1.weight = assign(
            block.ff.fc1.weight,
            params[f"model.layers.{l}.mlp.gate_proj.weight"],
            f"model.layers.{l}.mlp.gate_proj.weight"
        )
        block.ff.fc2.weight = assign(
            block.ff.fc2.weight,
            params[f"model.layers.{l}.mlp.up_proj.weight"],
            f"model.layers.{l}.mlp.up_proj.weight"
        )
        block.ff.fc3.weight = assign(
            block.ff.fc3.weight,
            params[f"model.layers.{l}.mlp.down_proj.weight"],
            f"model.layers.{l}.mlp.down_proj.weight"
        )
        block.norm2.scale = assign(
            block.norm2.scale,
            params[f"model.layers.{l}.post_attention_layernorm.weight"],
            f"model.layers.{l}.post_attention_layernorm.weight"
        )

    # Final normalization and output head
    model.final_norm.scale = assign(model.final_norm.scale, params["model.norm.weight"], "model.norm.weight")

    if "lm_head.weight" in params:
        model.out_head.weight = assign(model.out_head.weight, params["lm_head.weight"], "lm_head.weight")
    else:
        # Model uses weight tying, hence we reuse the embedding layer weights here
        print("Model uses weight tying.")
        model.out_head.weight = assign(model.out_head.weight, params["model.embed_tokens.weight"],
"model.embed_tokens.weight")


class Qwen3Tokenizer:
    _SPECIALS = [
        "<|endoftext|>",
        "<|im_start|>", "<|im_end|>",
```

```
        "<|object_ref_start|>", "<|object_ref_end|>",
        "<|box_start|>", "<|box_end|>",
        "<|quad_start|>", "<|quad_end|>",
        "<|vision_start|>", "<|vision_end|>",
        "<|vision_pad|>", "<|image_pad|>", "<|video_pad|>",
    ]
    _SPLIT_RE = re.compile(r"(<\|[^>]+?\|>)")

    def __init__(self, tokenizer_file_path="tokenizer.json", repo_id=None,
                 apply_chat_template=True, add_generation_prompt=False, add_thinking=False):

        self.apply_chat_template = apply_chat_template
        self.add_generation_prompt = add_generation_prompt
        self.add_thinking = add_thinking

        tok_file = Path(tokenizer_file_path)
        self._tok = Tokenizer.from_file(str(tok_file))
        self._special_to_id = {t: self._tok.token_to_id(t) for t in self._SPECIALS}

        self.pad_token_id = self._special_to_id.get("<|endoftext|>")
        self.eos_token_id = self.pad_token_id

        if repo_id and "Base" not in repo_id:
            eos_token = "<|im_end|>"
        else:
            eos_token = "<|endoftext|>"
        if eos_token in self._special_to_id:
            self.eos_token_id = self._special_to_id[eos_token]

    def encode(self, text, chat_wrapped=None):
        if chat_wrapped is None:
            chat_wrapped = self.apply_chat_template

        stripped = text.strip()
        if stripped in self._special_to_id and "\n" not in stripped:
            return [self._special_to_id[stripped]]

        if chat_wrapped:
            text = self._wrap_chat(text)

        ids = []
        for part in filter(None, self._SPLIT_RE.split(text)):
            if part in self._special_to_id:
                ids.append(self._special_to_id[part])
            else:
                ids.extend(self._tok.encode(part).ids)
        return ids
```

```python

    def decode(self, ids):
        return self._tok.decode(ids, skip_special_tokens=False)

    def _wrap_chat(self, user_msg):
        s = f"<|im_start|>user\n{user_msg}<|im_end|>\n"
        if self.add_generation_prompt:
            s += "<|im_start|>assistant"
            if self.add_thinking:
                s += "\n"
            else:
                s += "\n<think>\n\n</think>\n\n"
        return s


def generate_text_basic_stream(model, token_ids, max_new_tokens, eos_token_id=None):
    model.eval()
    with torch.no_grad():
        for _ in range(max_new_tokens):
            out = model(token_ids)[:, -1]
            next_token = torch.argmax(out, dim=-1, keepdim=True)

            if (eos_token_id is not None
                    and torch.all(next_token == eos_token_id)):
                break

            yield next_token

            token_ids = torch.cat([token_ids, next_token], dim=1)


def model_memory_size(model, input_dtype=torch.float32):
    total_params = 0
    total_grads = 0
    for param in model.parameters():
        # Calculate total number of elements per parameter
        param_size = param.numel()
        total_params += param_size
        # Check if gradients are stored for this parameter
        if param.requires_grad:
            total_grads += param_size

    # Calculate buffer size (non-parameters that require memory)
    total_buffers = sum(buf.numel() for buf in model.buffers())

    # Size in bytes = (Number of elements) * (Size of each element in bytes)
    # We assume parameters and gradients are stored in the same type as input dtype
    element_size = torch.tensor(0, dtype=input_dtype).element_size()
```

```python
    total_memory_bytes = (total_params + total_grads + total_buffers) * element_size

    # Convert bytes to gigabytes
    total_memory_gb = total_memory_bytes / (1024 ** 3)

    return total_memory_gb


if __name__ == "__main__":
    # loading model
    USE_REASONING_MODEL = True
    choose_model = "0.6B"
    qwen3_cfg = set_qwen3_config(choose_model)

    model = Qwen3Model(qwen3_cfg)

    print(f"model: \n{model}")

    model(torch.tensor([1, 2, 3]).unsqueeze(0))

    # calculate model size
    total_params = sum(p.numel() for p in model.parameters())
    print(f"Total number of parameters: {total_params:,}")

    # Account for weight tying
    total_params_normalized = total_params - model.tok_emb.weight.numel()
    print(f"\nTotal number of unique parameters: {total_params_normalized:,}")

    print(f"float32 (PyTorch default): {model_memory_size(model, input_dtype=torch.float32):.2f} GB")
    print(f"bfloat16: {model_memory_size(model, input_dtype=torch.bfloat16):.2f} GB")

    # set gpu device and load model to device
    if torch.cuda.is_available():
        device = torch.device("cuda")
    elif torch.backends.mps.is_available():
        device = torch.device("mps")
    else:
        device = torch.device("cpu")

    model.to(device)

    # download weights
    if USE_REASONING_MODEL:
        repo_id = f"Qwen/Qwen3-{choose_model}"
    else:
        repo_id = f"Qwen/Qwen3-{choose_model}-Base"

```

```python
    local_dir = Path(repo_id).parts[-1]

    if choose_model == "0.6B":
        weights_file = hf_hub_download(
            repo_id=repo_id,
            filename="model.safetensors",
            local_dir=local_dir,
        )
        tokenizer_file = hf_hub_download(
            repo_id=repo_id,
            filename="tokenizer.json",
            local_dir=local_dir,
        )

        weights_dict = load_file(weights_file)

    else:
        repo_dir = snapshot_download(repo_id=repo_id, local_dir=local_dir)
        index_path = os.path.join(repo_dir, "model.safetensors.index.json")
        with open(index_path, "r") as f:
            index = json.load(f)

        weights_dict = {}
        for filename in set(index["weight_map"].values()):
            shard_path = os.path.join(repo_dir, filename)
            shard = load_file(shard_path)
            weights_dict.update(shard)

    load_weights_into_qwen(model, qwen3_cfg, weights_dict)
    model.to(device)
    del weights_dict

    # download tokenizer
    if USE_REASONING_MODEL:
        tokenizer_file_path = f"Qwen3-{choose_model}/tokenizer.json"
    else:
        tokenizer_file_path = f"Qwen3-{choose_model}-Base/tokenizer.json"

    tokenizer = Qwen3Tokenizer(
        tokenizer_file_path=tokenizer_file_path,
        repo_id=repo_id,
        add_generation_prompt=USE_REASONING_MODEL,
        add_thinking=USE_REASONING_MODEL
    )

    # generate text
    prompt = "请给我简要的介绍下大模型."

```

```python
    input_token_ids = tokenizer.encode(prompt)
    text = tokenizer.decode(input_token_ids)
    print(f'text: {text}')

    input_token_ids_tensor = torch.tensor(input_token_ids, device=device).unsqueeze(0)
    start_time = time.perf_counter()
    llm_response = ""
    token_num = 0
    for token in generate_text_basic_stream(
            model=model,
            token_ids=input_token_ids_tensor,
            max_new_tokens=500,
            eos_token_id=tokenizer.eos_token_id
    ):
        token_id = token.squeeze(0).tolist()
        current_token = tokenizer.decode(token_id)
        llm_response += current_token
        token_num += 1
        print(
            current_token,
            end="",
            flush=True
        )
    end_time = time.perf_counter()
    print(f"共生成token数: {token_num}, 耗时: {end_time - start_time}, tps: {token_num / (end_time - start_time)}")
    print(f"中文字符数: {len(current_token)}, 耗时: {end_time - start_time}, 每秒中文字符数: {len(current_token) / (end_time - start_time)}")

```

- 输出示例

```
model:
Qwen3Model(
  (tok_emb): Embedding(151936, 1024)
  (trf_blocks): ModuleList(
    (0-27): 28 x TransformerBlock(
      (att): GroupQueryAttention(
        (W_query): Linear(in_features=1024, out_features=2048, bias=False)
        (W_key): Linear(in_features=1024, out_features=1024, bias=False)
        (W_value): Linear(in_features=1024, out_features=1024, bias=False)
        (out_proj): Linear(in_features=2048, out_features=1024, bias=False)
        (q_norm): RMSNorm()
        (k_norm): RMSNorm()
      )
      (ff): FeedForward(
```

```
        (fc1): Linear(in_features=1024, out_features=3072, bias=False)
        (fc2): Linear(in_features=1024, out_features=3072, bias=False)
        (fc3): Linear(in_features=3072, out_features=1024, bias=False)
      )
      (norm1): RMSNorm()
      (norm2): RMSNorm()
    )
  )
  (final_norm): RMSNorm()
  (out_head): Linear(in_features=1024, out_features=151936, bias=False)
)


Total number of parameters: 751,632,384

Total number of unique parameters: 596,049,920

float32 (PyTorch default): 5.64 GB
bfloat16: 2.82 GB


text: <|im_start|>user
请给我简要的介绍下大模型.<|im_end|>

<|im_start|>assistant

<think>
好的，用户让我简要介绍大模型。首先，我需要确定用户的需求是什么。他们可能对大模型的基本概念、应用场景感兴趣，或者需要了解其特点和优势。用户可能没有太多背景
，所以需要保持回答简洁明了。
接下来，我应该从几个关键点来组织信息：大模型的定义、核心特点、应用场景、优势和挑战。要确保每个部分都简短，避免冗长。同时，要使用用户容易理解的语言，避免专业术语过多。

还要考虑用户可能的深层需求。他们可能想了解大模型的实际应用，或者是否需要进一步的细节。因此，在介绍时，可以提到一些实际例子，比如医疗、金融、教育等，这样用
户更容易理解。

另外，要注意结构清晰，分点说明，这样用户一目了然。可能需要检查是否有遗漏的重要信息，比如数据来源、训练方法等，但保持简要。最后，确保回答友好，鼓励用户进一
步提问。
</think>

大模型（Large Language Model，LLM）是一种基于深度学习的AI系统，能够理解和生成人类语言。它通过大量文本数据训练，具备理解、生成和推理能力，广泛应用于自然语
言处理（NLP）领域。核心特点包括但不限于：

1. **语言理解**：理解上下文、语法和语义，支持多轮对话和复杂推理。
```

```markdown
2. **生成能力**：生成文本、代码、摘要等，适用于写作、内容创作等场景。
3. **多模态处理**：支持文本、图像、音频等多种信息的交互。

**优势**：高效处理复杂任务，提升效率；**挑战**：数据隐私、伦理问题、模型可解释性等仍需关注。

```

2 从零开始实现一个Qwen3(MoE)

2.1 简介

- 实现一个Qwen3-MoE架构，Qwen3 MoE模型与dense模型共享基本架构，所以这里除了MOE架构的代码，其他基本和**dense的实现**一致。

  另外这里是在一张H20上去跑的，实际显存用了67G左右，因为原作者实现的这个，可以支持30B-A3B，同时也支持235B-A22B(可能需要4张H20/H100/A100)，所以本章下面的，都是针对Qwen3-30B-A3B模型。

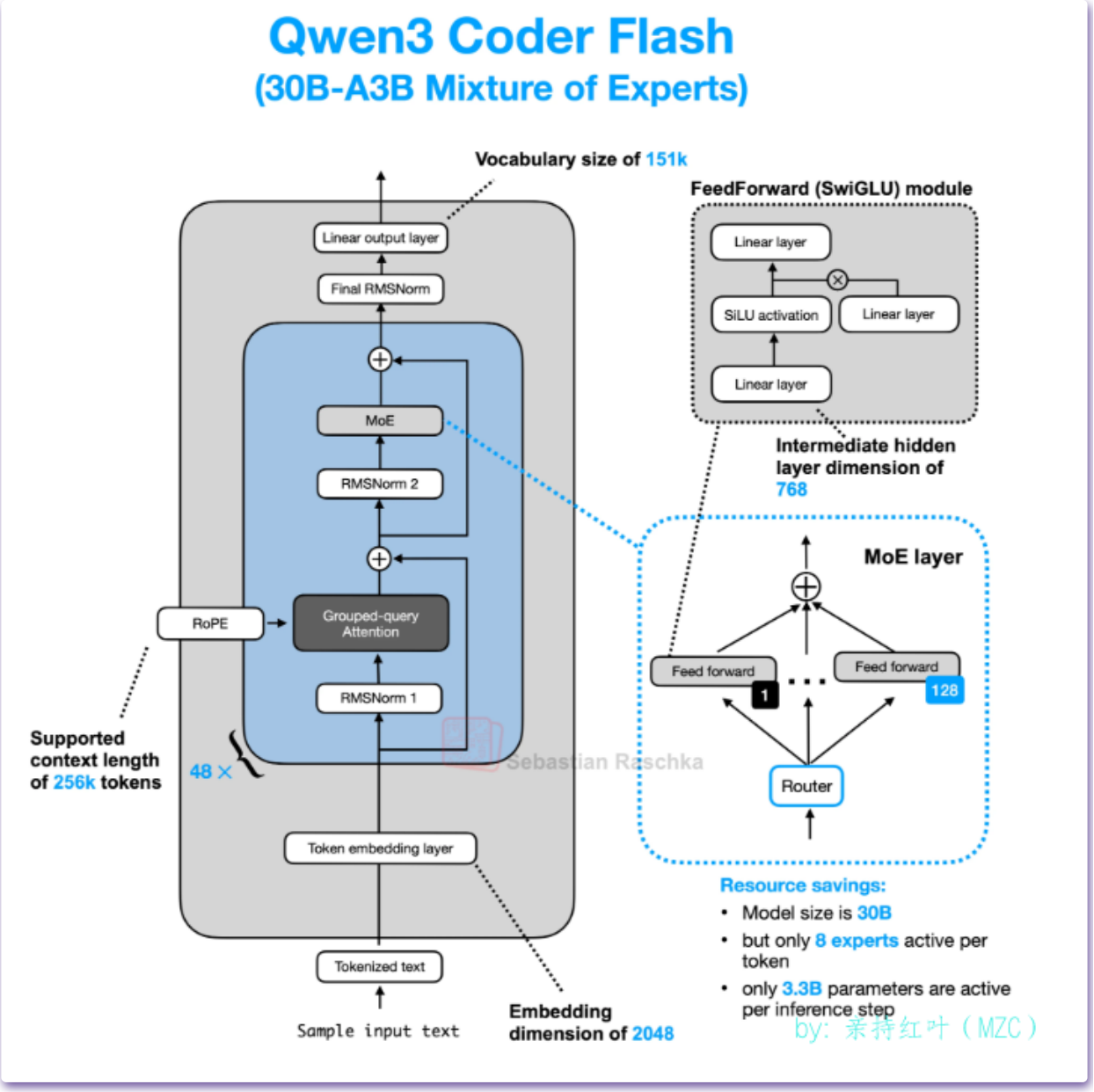
2.1.1 Qwen3 MoE架构

- 我们结合**Qwen/Qwen3-30B-A3B-Instruct-2507**的配置参数

```
{
  "architectures": [
    "Qwen3MoeForCausalLM"
  ],
  "attention_bias": false,
  "attention_dropout": 0.0,
  "bos_token_id": 151643,
  "decoder_sparse_step": 1,
  "eos_token_id": 151645,
  "head_dim": 128,
  "hidden_act": "silu",
  "hidden_size": 2048,
  "initializer_range": 0.02,
  "intermediate_size": 6144,
  "max_position_embeddings": 262144,
  "max_window_layers": 48,
  "mlp_only_layers": [],
  "model_type": "qwen3_moe",
  "moe_intermediate_size": 768,
  "norm_topk_prob": true,
  "num_attention_heads": 32,
  "num_experts": 128,
  "num_experts_per_tok": 8,
  "num_hidden_layers": 48,
  "num_key_value_heads": 4,
  "output_router_logits": false,
  "rms_norm_eps": 1e-06,
  "rope_scaling": null,
  "rope_theta": 10000000,
  "router_aux_loss_coef": 0.001,
  "sliding_window": null,
  "tie_word_embeddings": false,
  "torch_dtype": "bfloat16",
  "transformers_version": "4.51.0",
```

```
    "use_cache": true,
    "use_sliding_window": false,
    "vocab_size": 151936
  }
```

以及下面的图



备注： Qwen3-Coder-30B-A3B-Instruct 被称为Qwen3 Coder Flash，原作者这里使用的是Qwen3 Coder Flash，但是这里我使用的是Qwen3-30B-A3B-Instruct-2507，不影响，架构一样的，另外moe架构的Decoder中也是使用了qk_norm的，这里原作者没有画。

先来简单的看一下Qwen3-MoE系列的整体架构

1. 词汇表大小： 151936
2. 训练的上下文长度： 262144
3. MoE专家数量：128
4. 每个token使用的专家数：8
5. MoE中间层大小：768

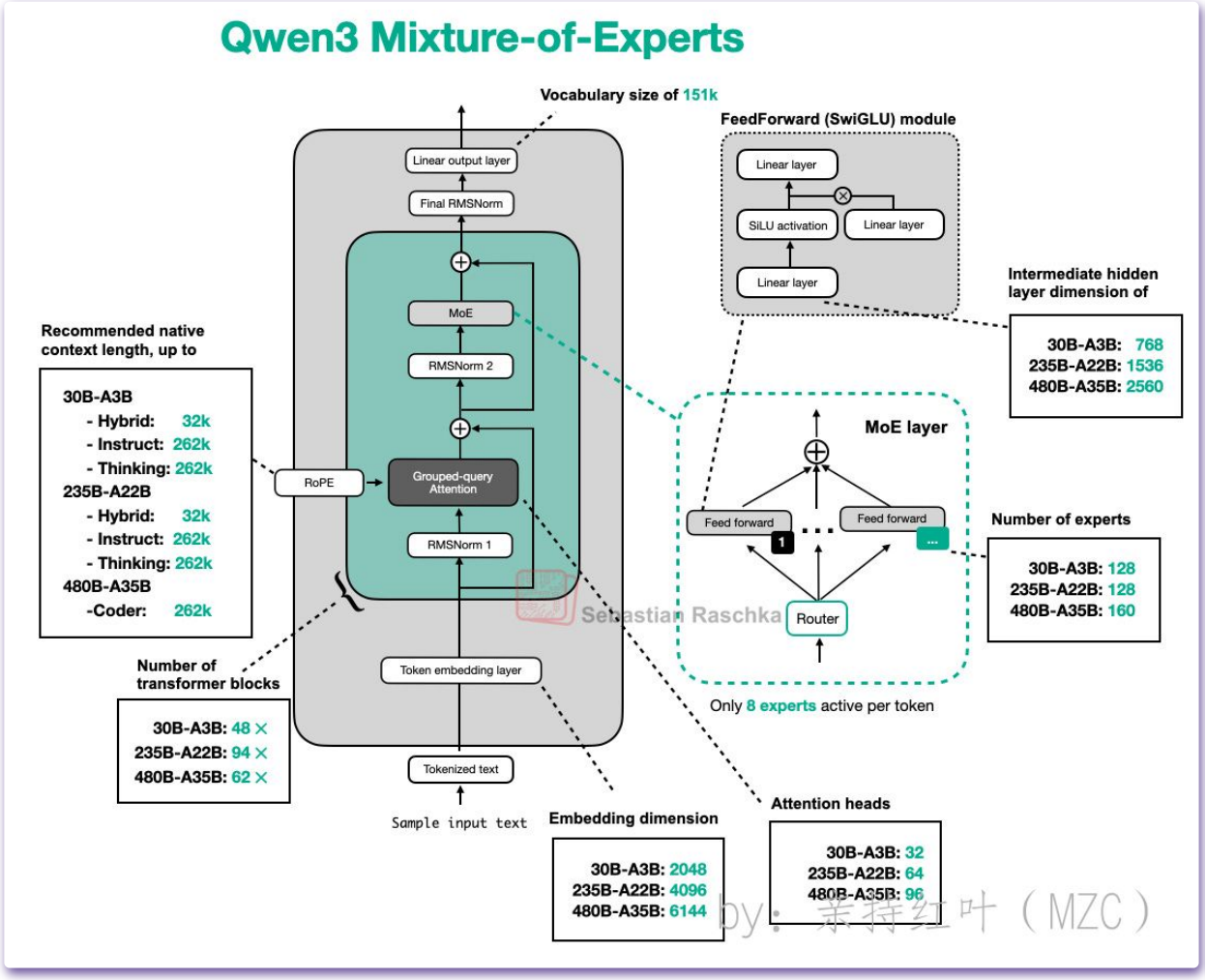
6. 激活函数：SwiGLU （Swish/SiLU + GLU）
7. 使用ROPE位置编码
8. 使用Pre-Norm，且Normalization函数为RMSNorm
9. 使用GQA, q共32个头，G=4

使用pydantic定义一个config的数据模型，后续代码中出现的cfg即是这个数据模型的实例化

```python
import torch
from pydantic import BaseModel, Field

class Qwen3MoeConfig(BaseModel):
    """Pydantic model for Qwen3 moe configuration"""
    model_config = {"arbitrary_types_allowed": True}

    vocab_size: int = Field(..., description="词汇表大小")
    context_length: int = Field(..., description="用于训练模型的上下文长度")
    emb_dim: int = Field(..., description="Embedding 维度")
    n_heads: int = Field(..., description="注意力头的数量")
    n_layers: int = Field(..., description="层数")
    head_dim: int = Field(..., description="GQA 中每个注意力头的维度大小")
    qk_norm: bool = Field(..., description="是否在 GQA 中对key和value进行归一化")
    n_kv_groups: int = Field(..., description="用于分组查询注意力的 KV 组数")
    rope_base: float = Field(..., description="RoPE 中'theta'的基数值")
    dtype: torch.dtype = Field(..., description="较低精度的数据类型，用于降低显存占用")
    # MoE 相关参数
    num_experts: int = Field(..., description="专家数量")
    num_experts_per_tok: int = Field(..., description="每个token使用的专家数量")
    moe_intermediate_size: int = Field(..., description="MoE中间层大小")

QWEN3_CONFIG = {
      "vocab_size": 151_936,
      "context_length": 262_144,
      "emb_dim": 2048,
      "n_heads": 32,
      "n_layers": 48,
      "head_dim": 128,
      "qk_norm": True,
      "n_kv_groups": 4,
      "rope_base": 10_000_000.0,
      "dtype": torch.bfloat16,
      "num_experts": 128,
      "num_experts_per_tok": 8,
      "moe_intermediate_size": 768,
```

```
    }

cfg = Qwen3MoeConfig(**QWEN3_CONFIG)
```

- 备注，下面是各不同的Qwen3-moe模型的架构图



2.1.2 参考

- **参考代码**：

> https://github.com/rasbt/LLMs-from-scratch/blob/main/ch05/11_qwen3/standalone-qwen3-moe.ipynb

- Qwen3: Think Deeper, Act Faster

> https://qwenlm.github.io/blog/qwen3/

- Qwen3 Technical Report Qwen3 技术报告

> https://arxiv.org/abs/2505.09388

2.2 结构

2.2.1 RMSNorm（Root Mean Square Layer Normalization）

2.2.1.1 RMSNorm介绍

- **transformer-RMSNorm**
- RMSNorm通过对神经元输出的均方根值进行缩放，在保持训练稳定性的同时降低计算复杂度。与LayerNorm相比，RMSNorm移除了均值中心化操作，仅通过均方根值进行缩放，减少了约30%的计算量。

  在Qwen3中，RMSNorm被广泛应用于每个Transformer层的归一化操作，包括注意力层前后和前馈网络前后的归一化。该方法通过单一缩放参数调整特征幅度，避免对输入进行平移操作，特别适合大规模模型训练。

2.2.1.2 数学公式

- RMSNorm对输入张量最后一个维度进行均方根标准化：

$$
\begin{aligned}
\mathrm{RMS}(x) &= \sqrt{\frac{1}{emb_dim}\sum_{i=1}^{emb_dim} x_i^2} \\
\mathrm{out} &= \gamma \cdot \frac{x}{\sqrt{\mathrm{RMS}(x)^2+\epsilon}} + \beta
\end{aligned}
\tag{10}
$$

  其中：

  - $\gamma \in \mathbb{R}^{emb_dim}$：可学习缩放参数（初始化为1）
  - β： 科学系平移参数（初始化为0）
  - ϵ：数值稳定系数（默认1e-6）

  emb_dim 为特征维度。

2.2.1.3 代码实现

- Qwen3中的RMSNorm实现

```python
class RMSNorm(nn.Module):
    def __init__(self, emb_dim, eps=1e-6, bias=False, qwen3_compatible=True):
        super().__init__()
        self.eps = eps
        self.qwen3_compatible = qwen3_compatible
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim)) if bias else None

    def forward(self, x):
        input_dtype = x.dtype

```

```
        if self.qwen3_compatible:
            x = x.to(torch.float32)

        variance = x.pow(2).mean(dim=-1, keepdim=True)
        norm_x = x * torch.rsqrt(variance + self.eps)
        norm_x = norm_x * self.scale

        if self.shift is not None:
            norm_x = norm_x + self.shift

        return norm_x.to(input_dtype)
```

2.2.2 Rotary Position Embedding (RoPE)

2.2.2.1 RoPE介绍

- **transformer-旋转位置编码RoPE**
- 旋转位置编码（Rotary Position Embedding, RoPE）是Qwen3模型中使用的位置编码方法，通过对查询和键向量进行旋转变换来注入位置信息。与传统的绝对位置编码不同，RoPE能够更好地处理相对位置关系，并且对序列长度具有更好的外推能力。

  在Qwen3中，RoPE被应用于每个注意力头的查询和键向量，通过预计算的正弦和余弦值对向量进行旋转变换，使得模型能够感知token之间的相对位置关系。

2.2.2.2 RoPE的数学公式

- RoPE通过复数域的旋转操作实现位置编码：

  对于位置为m的向量$\mathbf{x}$，RoPE变换定义为：

$$f(\mathbf{x}, m) = \mathbf{R}_{\Theta,m}\mathbf{x} \tag{11}$$

  其中旋转矩阵$\mathbf{R}_{\Theta,m}$的计算公式为：

$$\mathbf{R}_{\Theta,m} = \begin{pmatrix} \cos(m\theta_0) & -\sin(m\theta_0) & 0 & 0 & \cdots \\ \sin(m\theta_0) & \cos(m\theta_0) & 0 & 0 & \cdots \\ 0 & 0 & \cos(m\theta_1) & -\sin(m\theta_1) & \cdots \\ 0 & 0 & \sin(m\theta_1) & \cos(m\theta_1) & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix} \tag{12}$$

  其中$\theta_i = \text{base}^{-2i/d}$，在Qwen3-MoE中base=10,000,000。

2.2.2.3 代码实现

- RoPE参数计算函数

```
def compute_rope_params(head_dim, theta_base=10_000, context_length=4096, dtype=torch.float32):
    assert head_dim % 2 == 0, "Embedding dimension must be even"

```

```python
    # 计算逆频率
    inv_freq = 1.0 / (theta_base ** (torch.arange(0, head_dim, 2, dtype=dtype)[: (head_dim // 2)].float() / head_dim))

    # 生成位置索引
    positions = torch.arange(context_length, dtype=dtype)

    # 计算角度
    angles = positions[:, None] * inv_freq[None, :]  # Shape: (context_length, head_dim // 2)

    # 扩展角度以匹配head_dim
    angles = torch.cat([angles, angles], dim=1)  # Shape: (context_length, head_dim)

    # 预计算正弦和余弦
    cos = torch.cos(angles)
    sin = torch.sin(angles)

    return cos, sin
```

- RoPE应用函数

```python
def apply_rope(x, cos, sin):
    # x: (batch_size, num_heads, seq_len, head_dim)
    batch_size, num_heads, seq_len, head_dim = x.shape
    assert head_dim % 2 == 0, "Head dimension must be even"

    # 将x分为前半部分和后半部分
    x1 = x[..., : head_dim // 2]  # 前半部分
    x2 = x[..., head_dim // 2 :]  # 后半部分

    # 调整sin和cos的形状
    cos = cos[:seq_len, :].unsqueeze(0).unsqueeze(0)  # Shape: (1, 1, seq_len, head_dim)
    sin = sin[:seq_len, :].unsqueeze(0).unsqueeze(0)

    # 应用旋转变换
    rotated = torch.cat((-x2, x1), dim=-1)
    x_rotated = (x * cos) + (rotated * sin)

    # 使用与输入相同的精度
    return x_rotated.to(dtype=x.dtype)
```

2.2.3 Grouped Query Attention (GQA)

2.2.3.1 GQA介绍

- **MHA、MQA、GQA：大模型注意力机制的演进**
- 分组查询注意力（Grouped Query Attention, GQA）是Qwen3模型中使用的高效注意力机

制。GQA通过减少键值对（Key-Value）的数量来降低内存使用和计算复杂度，同时保持与多头注意力相近的性能。

在传统的多头注意力中，每个头都有独立的查询、键、值投影。而在GQA中，多个查询头共享同一组键值对，通过分组的方式实现计算效率的提升。Qwen3-30B-A3B使用4个KV组来支持32个注意力头。

2.2.3.2 数学公式

- GQA的计算过程可以表示为：

$$\mathrm{GQA}(X) = \mathrm{Concat}(\mathrm{head}_1, \mathrm{head}_2, \ldots, \mathrm{head}_h)W^O \tag{13}$$

其中每个头的计算为：

$$\mathrm{head}_i = \mathrm{Attention}(XW_i^Q, XW_{\lfloor i/g \rfloor}^K, XW_{\lfloor i/g \rfloor}^V) \tag{14}$$

关键参数：

  - h：查询头的总数
  - g：每组的大小（group_size = num_heads / num_kv_groups）
  - $\lfloor i/g \rfloor$：第i个查询头对应的KV组索引

2.2.3.3 代码实现

- Qwen3中的GQA实现

```python
class GroupedQueryAttention(nn.Module):
    def __init__(
        self, d_in, num_heads, num_kv_groups, head_dim=None, qk_norm=False, dtype=None
    ):
        super().__init__()
        assert num_heads % num_kv_groups == 0, "num_heads must be divisible by num_kv_groups"

        self.num_heads = num_heads
        self.num_kv_groups = num_kv_groups
        self.group_size = num_heads // num_kv_groups

        if head_dim is None:
            assert d_in % num_heads == 0, "`d_in` must be divisible by `num_heads` if `head_dim` is not set"
            head_dim = d_in // num_heads

        self.head_dim = head_dim
        self.d_out = num_heads * head_dim

        self.W_query = nn.Linear(d_in, self.d_out, bias=False, dtype=dtype)
        self.W_key = nn.Linear(d_in, num_kv_groups * head_dim, bias=False, dtype=dtype)
```

```python
        self.W_value = nn.Linear(d_in, num_kv_groups * head_dim, bias=False, dtype=dtype)

        self.out_proj = nn.Linear(self.d_out, d_in, bias=False, dtype=dtype)

        if qk_norm:
            self.q_norm = RMSNorm(head_dim, eps=1e-6)
            self.k_norm = RMSNorm(head_dim, eps=1e-6)
        else:
            self.q_norm = self.k_norm = None

    def forward(self, x, mask, cos, sin):
        b, num_tokens, _ = x.shape

        # 应用投影
        queries = self.W_query(x)  # (b, num_tokens, num_heads * head_dim)
        keys = self.W_key(x)       # (b, num_tokens, num_kv_groups * head_dim)
        values = self.W_value(x)   # (b, num_tokens, num_kv_groups * head_dim)

        # 重塑
        queries = queries.view(b, num_tokens, self.num_heads, self.head_dim).transpose(1, 2)
        keys = keys.view(b, num_tokens, self.num_kv_groups, self.head_dim).transpose(1, 2)
        values = values.view(b, num_tokens, self.num_kv_groups, self.head_dim).transpose(1, 2)

        # 可选的归一化
        if self.q_norm:
            queries = self.q_norm(queries)
        if self.k_norm:
            keys = self.k_norm(keys)

        # 应用RoPE
        queries = apply_rope(queries, cos, sin)
        keys = apply_rope(keys, cos, sin)

        # 扩展K和V以匹配头的数量
        keys = keys.repeat_interleave(self.group_size, dim=1)
        values = values.repeat_interleave(self.group_size, dim=1)

        # 注意力
        attn_scores = queries @ keys.transpose(2, 3)
        attn_scores = attn_scores.masked_fill(mask, -torch.inf)
        attn_weights = torch.softmax(attn_scores / self.head_dim**0.5, dim=-1)

```

```
        context = (attn_weights @ values).transpose(1, 2).reshape(b, num_tokens, self.d_out)
        return self.out_proj(context)
```

- 关键特性解析

特性	说明	Qwen3-30B-A3B配置
num_heads	查询头数量	32
num_kv_groups	键值组数量	4
group_size	每组大小	8
head_dim	每个头的维度	128
qk_norm	QK归一化	True

- 维度变化流程

操作步骤	张量形状变化示例
输入数据	[batch_size, seq_len, emb_dim]
查询投影	[batch_size, seq_len, num_heads * head_dim]
键值投影	[batch_size, seq_len, num_kv_groups * head_dim]
重塑查询	[batch_size, num_heads, seq_len, head_dim]
重塑键值	[batch_size, num_kv_groups, seq_len, head_dim]
扩展键值	[batch_size, num_heads, seq_len, head_dim]
注意力计算	[batch_size, num_heads, seq_len, head_dim]
输出投影	[batch_size, seq_len, emb_dim]

2.2.4 MoE前馈网络

2.2.4.1 MoE介绍

- MoE (Mixture of Experts) 是一种稀疏激活的神经网络架构，其核心思想是不让所有参数都参与每个token的计算，而是为每个token动态选择一部分"专家"(experts)进行计算。这种方法可以显著增加模型参数量，同时保持计算效率。

  在Qwen3-MoE中，每个token通过门控网络选择最适合的8个专家（从128个专家中），然后将这些专家的输出进行加权组合。这种稀疏激活机制使得模型能够拥有更大的参数量，同时保持合理的计算成本。

2.2.4.2 数学公式

- MoE前馈网络的计算过程可以表示为：

$$y = \sum_{i=1}^{E} G(x)_i \cdot E_i(x) \tag{15}$$

其中：

  - E 是专家总数
  - $G(x)_i$ 是门控网络为第i个专家分配的权重
  - $E_i(x)$ 是第i个专家对输入x的处理结果

在Qwen3-MoE中，门控网络选择top-k个专家（k=8），并对它们的输出进行加权求和。

2.2.4.3 代码实现

- Qwen3中的MoE前馈网络实现

```python
class MoEFeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.num_experts_per_tok = cfg.num_experts_per_tok
        self.num_experts = cfg.num_experts
        self.gate = nn.Linear(cfg.emb_dim, cfg.num_experts, bias=False, dtype=cfg.dtype)

        # meta device用于减少初始化模型时的内存压力
        meta_device = torch.device("meta")
        self.fc1 = nn.ModuleList([
            nn.Linear(
                cfg.emb_dim, cfg.moe_intermediate_size,
                bias=False, dtype=cfg.dtype, device=meta_device)
            for _ in range(cfg.num_experts)]
        )
        self.fc2 = nn.ModuleList([
            nn.Linear(
                cfg.emb_dim, cfg.moe_intermediate_size,
                bias=False, dtype=cfg.dtype, device=meta_device
                )
            for _ in range(cfg.num_experts)]
        )
        self.fc3 = nn.ModuleList([
            nn.Linear(
                cfg.moe_intermediate_size, cfg.emb_dim,
                bias=False, dtype=cfg.dtype, device=meta_device
                )
            for _ in range(cfg.num_experts)]
        )

    def forward(self, x):
        b, seq_len, embed_dim = x.shape
        scores = self.gate(x)  # (b, seq_len, num_experts)
```

```python
        topk_scores, topk_indices = torch.topk(scores, self.num_experts_per_tok, dim=-1)
        topk_probs = torch.softmax(topk_scores, dim=-1)

        expert_outputs = []
        for e in range(self.num_experts):
            hidden = torch.nn.functional.silu(self.fc1[e](x)) * self.fc2[e](x)
            out = self.fc3[e](hidden)
            expert_outputs.append(out.unsqueeze(-2))
        expert_outputs = torch.cat(expert_outputs, dim=-2)  # (b, t, num_experts, emb_dim)

        gating_probs = torch.zeros_like(scores)

        for i in range(self.num_experts_per_tok):
            indices = topk_indices[..., i:i+1]
            prob = topk_probs[..., i:i+1]
            gating_probs.scatter_(dim=-1, index=indices, src=prob)
        gating_probs = gating_probs.unsqueeze(-1)  # (b, t, num_experts, 1)

        # 对专家输出进行加权求和
        y = (gating_probs * expert_outputs).sum(dim=-2)
        return y
```

- 关键特性解析
  1. `门控机制`：
     - **门控网络**: 通过线性层将输入映射到专家得分
     - **Top-k选择**: 选择得分最高的k个专家
     - **软门控**: 使用softmax对选中专家的得分进行归一化
  2. `专家网络`：
     - **SwiGLU激活**: 每个专家使用SwiGLU激活函数
     - **Meta设备**: 初始化时使用meta设备减少内存压力
     - **模块化设计**: 使用ModuleList管理多个专家
- 维度变化流程

操作步骤	张量形状变化示例	Qwen3-30B-A3B配置
输入数据	[batch_size, seq_len, emb_dim]	[4, 100, 2048]
门控网络	[batch_size, seq_len, num_experts]	[4, 100, 128]
Top-k选择	[batch_size, seq_len, num_experts_per_tok]	[4, 100, 8]
专家计算	[batch_size, seq_len, num_experts, emb_dim]	[4, 100, 128, 2048]
门控概率	[batch_size, seq_len, num_experts, 1]	[4, 100, 128, 1]
加权求和	[batch_size, seq_len, emb_dim]	[4, 100, 2048]

2.2.5 标准前馈网络

- 代码：

```
class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.fc1 = nn.Linear(cfg.emb_dim, cfg.hidden_dim, dtype=cfg.dtype, bias=False)
        self.fc2 = nn.Linear(cfg.emb_dim, cfg.hidden_dim, dtype=cfg.dtype, bias=False)
        self.fc3 = nn.Linear(cfg.hidden_dim, cfg.emb_dim, dtype=cfg.dtype, bias=False)

    def forward(self, x):
        x_fc1 = self.fc1(x)
        x_fc2 = self.fc2(x)
        x = nn.functional.silu(x_fc1) * x_fc2
        return self.fc3(x)
```

- 这个标准前馈网络使用SwiGLU激活函数，与MoE中每个专家使用的激活函数相同。

2.2.6 Transformer Block

2.2.6.1 Transformer Block介绍

- Qwen3-MoE的Transformer Block是模型的核心构建单元，每个Block包含一个分组查询注意力层和一个MoE前馈网络层（或标准前馈网络层）。与标准Transformer不同，Qwen3采用了Pre-Norm结构，即在每个子层之前应用RMSNorm，这种设计有助于训练稳定性和梯度流动。

  每个Transformer Block通过残差连接将输入与子层输出相加，形成深度网络中的信息高速公路。Qwen3-30B-A3B模型包含48个这样的Transformer Block，通过层层堆叠实现复杂的语言理解和生成能力。

2.2.6.2 代码实现

- Qwen3中的Transformer Block实现

```python
class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.att = GroupedQueryAttention(
            d_in=cfg.emb_dim,
            num_heads=cfg.n_heads,
            head_dim=cfg.head_dim,
            num_kv_groups=cfg.n_kv_groups,
            qk_norm=cfg.qk_norm,
            dtype=cfg.dtype
        )
        if cfg.num_experts > 0:
            self.ff = MoEFeedForward(cfg)
        else:
            self.ff = FeedForward(cfg)
        self.norm1 = RMSNorm(cfg.emb_dim, eps=1e-6)
        self.norm2 = RMSNorm(cfg.emb_dim, eps=1e-6)

    def forward(self, x, mask, cos, sin):
        # 注意力子层的残差连接
        shortcut = x
        x = self.norm1(x)
        x = self.att(x, mask, cos, sin)  # Shape [batch_size, num_tokens, emb_size]
        x = x + shortcut  # 添加原始输入

        # 前馈子层的残差连接
        shortcut = x
        x = self.norm2(x)
        x = self.ff(x)
        x = x + shortcut  # 添加原始输入

        return x
```

- 关键设计特点
  1. `条件选择前馈网络`：
     - 根据配置选择MoE前馈网络或标准前馈网络
     - 允许在同一架构中灵活切换稀疏和密集模型
  2. `Pre-Norm结构`：
     - 在每个子层之前应用归一化，而不是之后
     - 有助于训练稳定性，特别是在深层网络中
     - 减少梯度消失问题
  3. `残差连接`：
     - 每个子层都有直接的跳跃连接
     - 保证信息流动和梯度传播

- 使得深层网络训练成为可能

2.2.7 Qwen3-30B-A3B 完整模型

2.2.7.1 模型架构

- Qwen3-MoE的完整架构包含以下组件：
  1. **词嵌入层（Token Embedding）**：将输入token映射为高维向量
  2. **多层Transformer Block**：核心的特征提取和变换层
  3. **最终归一化层（Final Norm）**：输出前的最后一次归一化
  4. **输出投影层（Output Head）**：将隐藏状态映射到词汇表空间

  模型支持因果语言建模，通过上三角掩码确保生成过程中不会看到未来信息。

2.2.7.2 代码实现

- Qwen3-MoE完整模型实现

```python
class Qwen3MoeModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()

        # Main model parameters
        self.tok_emb = nn.Embedding(cfg.vocab_size, cfg.emb_dim, dtype=cfg.dtype)

        self.trf_blocks = nn.ModuleList(  # ModuleList since Sequential can only accept one input, and we need `x, mask, cos, sin`
            [TransformerBlock(cfg) for _ in range(cfg.n_layers)]
        )

        self.final_norm = RMSNorm(cfg.emb_dim)
        self.out_head = nn.Linear(cfg.emb_dim, cfg.vocab_size, bias=False, dtype=cfg.dtype)

        # Reusuable utilities
        if cfg.head_dim is None:
            head_dim = cfg.emb_dim // cfg.n_heads
        else:
            head_dim = cfg.head_dim
        cos, sin = compute_rope_params(
            head_dim=head_dim,
            theta_base=cfg.rope_base,
            context_length=cfg.context_length
        )
        self.register_buffer("cos", cos, persistent=False)
        self.register_buffer("sin", sin, persistent=False)
        self.cfg = cfg


```

```python
    def forward(self, in_idx):
        # Forward pass
        tok_embeds = self.tok_emb(in_idx)
        x = tok_embeds

        num_tokens = x.shape[1]
        mask = torch.triu(torch.ones(num_tokens, num_tokens, device=x.device, dtype=torch.bool), diagonal=1)

        for block in self.trf_blocks:
            x = block(x, mask, self.cos, self.sin)
        x = self.final_norm(x)
        logits = self.out_head(x.to(self.cfg.dtype))
        return logits
```

- 模型配置

```python
def set_qwen3_moe_config(repo_id):
    repo_id_list = ["Qwen/Qwen3-30B-A3B", "Qwen/Qwen3-235B-A22B-Instruct-2507",
                     "Qwen/Qwen3-30B-A3B-Thinking-2507", "Qwen/Qwen3-Coder-30B-A3B-Instruct",
                    "/root/Qwen/Qwen3-30B-A3B-Instruct-2507"]
    assert repo_id in repo_id_list, f"repo_id  error,must be one of {repo_id_list}"

    list_30B =  ["Qwen/Qwen3-30B-A3B","Qwen/Qwen3-30B-A3B-Thinking-2507", "Qwen/Qwen3-Coder-30B-A3B-Instruct",
                    "/root/Qwen/Qwen3-30B-A3B-Instruct-2507"]
    list_235B = ["Qwen/Qwen3-235B-A22B-Instruct-2507"]

    if repo_id in list_30B:
        # 30B
        QWEN3_CONFIG = {
                "vocab_size": 151_936,
                "context_length": 262_144,
                "emb_dim": 2048,
                "n_heads": 32,
                "n_layers": 48,
                "head_dim": 128,
                "qk_norm": True,
                "n_kv_groups": 4,
                "rope_base": 10_000_000.0,
                "dtype": torch.bfloat16,
                "num_experts": 128,
                "num_experts_per_tok": 8,
                "moe_intermediate_size": 768,
            }
    elif repo_id in list_235B:
        ## 235B
        QWEN3_CONFIG = {
                "vocab_size": 151_936,
```

```
                    "context_length": 262_144,
                    "emb_dim": 4096,
                    "n_heads": 64,
                    "n_layers": 94,
                    "head_dim": 128,
                    "qk_norm": True,
                    "n_kv_groups": 4,
                    "rope_base": 5_000_000.0,
                    "dtype": torch.bfloat16,
                    "num_experts": 128,
                    "num_experts_per_tok": 8,
                    "moe_intermediate_size": 1536,
                }
        else:
            raise ValueError(f"{repo_id} is not supported.")

        return Qwen3MoeConfig(**QWEN3_CONFIG)
```

- 不同规模模型对比

模型规模	参数量	嵌入维度	注意力头	层数	专家数量	每token专家数	MoE中间层大小
30B-A3B	30B/3B	2048	32	48	128	8	768
235B-A22B	235B/22B	4096	64	94	128	8	1536

- Qwen3-30B-A3B加载后的模型print

```
Qwen3MoeModel(
  (tok_emb): Embedding(151936, 2048)
  (trf_blocks): ModuleList(
    (0-47): 48 x TransformerBlock(
      (att): GroupedQueryAttention(
        (W_query): Linear(in_features=2048, out_features=4096, bias=False)
        (W_key): Linear(in_features=2048, out_features=512, bias=False)
        (W_value): Linear(in_features=2048, out_features=512, bias=False)
        (out_proj): Linear(in_features=4096, out_features=2048, bias=False)
        (q_norm): RMSNorm()
        (k_norm): RMSNorm()
      )
      (ff): MoEFeedForward(
        (gate): Linear(in_features=2048, out_features=128, bias=False)
        (fc1): ModuleList(
```

```
            (0-127): 128 x Linear(in_features=2048, out_features=768, bias=False)
          )
          (fc2): ModuleList(
            (0-127): 128 x Linear(in_features=2048, out_features=768, bias=False)
          )
          (fc3): ModuleList(
            (0-127): 128 x Linear(in_features=768, out_features=2048, bias=False)
          )
        )
        (norm1): RMSNorm()
        (norm2): RMSNorm()
      )
    )
    (final_norm): RMSNorm()
    (out_head): Linear(in_features=2048, out_features=151936, bias=False)
  )
```

2.3 文本生成+推理

2.3.1 文本生成

- 基础的贪婪解码算法：

```python
def generate_text_basic_stream(model, token_ids, max_new_tokens, eos_token_id=None):
    model.eval()
    with torch.no_grad():
        for _ in range(max_new_tokens):
            # 前向传播获取logits
            out = model(token_ids)[:, -1]  # 只取最后一个位置的输出

            # 贪婪解码：选择概率最高的token
            next_token = torch.argmax(out, dim=-1, keepdim=True)

            # 检查是否遇到结束token
            if (eos_token_id is not None
                    and torch.all(next_token == eos_token_id)):
               break

            yield next_token

            # 将新token添加到序列中
            token_ids = torch.cat([token_ids, next_token], dim=1)
```

2.3.2 Tokenizer实现

- Qwen3专用的Tokenizer实现

```python
class Qwen3Tokenizer:
    _SPECIALS = [
        "<|endoftext|>",
        "<|im_start|>", "<|im_end|>",
        "<|object_ref_start|>", "<|object_ref_end|>",
        "<|box_start|>", "<|box_end|>",
        "<|quad_start|>", "<|quad_end|>",
        "<|vision_start|>", "<|vision_end|>",
        "<|vision_pad|>", "<|image_pad|>", "<|video_pad|>",
    ]
    _SPLIT_RE = re.compile(r"(<\|[^>]+?\|>)")

    def __init__(self, tokenizer_file_path="tokenizer.json", repo_id=None,
                 apply_chat_template=True, add_generation_prompt=False, add_thinking=False):

        self.apply_chat_template = apply_chat_template
        self.add_generation_prompt = add_generation_prompt
        self.add_thinking = add_thinking

        tok_file = Path(tokenizer_file_path)
        self._tok = Tokenizer.from_file(str(tok_file))
        self._special_to_id = {t: self._tok.token_to_id(t) for t in self._SPECIALS}

        self.pad_token_id = self._special_to_id.get("<|endoftext|>")
        self.eos_token_id = self.pad_token_id

        if repo_id and "Base" not in repo_id:
            eos_token = "<|im_end|>"
        else:
            eos_token = "<|endoftext|>"
        if eos_token in self._special_to_id:
            self.eos_token_id = self._special_to_id[eos_token]

    def encode(self, text, chat_wrapped=None):
        if chat_wrapped is None:
            chat_wrapped = self.apply_chat_template

        stripped = text.strip()
        if stripped in self._special_to_id and "\n" not in stripped:
            return [self._special_to_id[stripped]]

        if chat_wrapped:
            text = self._wrap_chat(text)

        ids = []
```

```
        for part in filter(None, self._SPLIT_RE.split(text)):
            if part in self._special_to_id:
                ids.append(self._special_to_id[part])
            else:
                ids.extend(self._tok.encode(part).ids)
        return ids

    def decode(self, ids):
        return self._tok.decode(ids, skip_special_tokens=False)

    def _wrap_chat(self, user_msg):
        s = f"<|im_start|>user\n{user_msg}<|im_end|>\n"
        if self.add_generation_prompt:
            s += "<|im_start|>assistant"
            if self.add_thinking:
                s += "\n"
            else:
                s += "\n<think>\n\n</think>\n\n"
        return s
```

2.3.3 权重加载

- Qwen3-MoE模型权重加载函数

```
def load_weights_into_qwen(model, param_config, params):
    def assign(left, right, tensor_name="unknown"):
        if left.shape != right.shape:
            raise ValueError(f"Shape mismatch in tensor '{tensor_name}'. Left: {left.shape}, Right: {right.shape}")
        return torch.nn.Parameter(right.clone().detach() if isinstance(right, torch.Tensor) else torch.tensor(right))

    model.tok_emb.weight = assign(model.tok_emb.weight, params["model.embed_tokens.weight"], "model.embed_tokens.weight")

    for l in range(param_config.n_layers):
        block = model.trf_blocks[l]
        att = block.att

        # Q, K, V projections
        att.W_query.weight = assign(
            att.W_query.weight,
            params[f"model.layers.{l}.self_attn.q_proj.weight"],
            f"model.layers.{l}.self_attn.q_proj.weight"
        )
        att.W_key.weight = assign(
            att.W_key.weight,
            params[f"model.layers.{l}.self_attn.k_proj.weight"],
            f"model.layers.{l}.self_attn.k_proj.weight"
        )
        att.W_value.weight = assign(
            att.W_value.weight,
```

```
            params[f"model.layers.{l}.self_attn.v_proj.weight"],
            f"model.layers.{l}.self_attn.v_proj.weight"
        )

        # Output projection
        att.out_proj.weight = assign(
            att.out_proj.weight,
            params[f"model.layers.{l}.self_attn.o_proj.weight"],
            f"model.layers.{l}.self_attn.o_proj.weight"
        )

        # QK norms
        if hasattr(att, "q_norm") and att.q_norm is not None:
            att.q_norm.scale = assign(
                att.q_norm.scale,
                params[f"model.layers.{l}.self_attn.q_norm.weight"],
                f"model.layers.{l}.self_attn.q_norm.weight"
            )
        if hasattr(att, "k_norm") and att.k_norm is not None:
            att.k_norm.scale = assign(
                att.k_norm.scale,
                params[f"model.layers.{l}.self_attn.k_norm.weight"],
                f"model.layers.{l}.self_attn.k_norm.weight"
            )

        # Attention layernorm
        block.norm1.scale = assign(
            block.norm1.scale,
            params[f"model.layers.{l}.input_layernorm.weight"],
            f"model.layers.{l}.input_layernorm.weight"
        )

        # Feedforward weights
        if "num_experts" in param_config.model_dump():
            # 加载路由器（门控）权重
            block.ff.gate.weight = assign(
                block.ff.gate.weight,
                params[f"model.layers.{l}.mlp.gate.weight"],
                f"model.layers.{l}.mlp.gate.weight"
            )
            # 加载专家权重
            for e in range(param_config.num_experts):
                prefix = f"model.layers.{l}.mlp.experts.{e}"
                block.ff.fc1[e].weight = assign(
                    block.ff.fc1[e].weight,
                    params[f"{prefix}.gate_proj.weight"],
                    f"{prefix}.gate_proj.weight"
                )
                block.ff.fc2[e].weight = assign(
                    block.ff.fc2[e].weight,
```

```
                    params[f"{prefix}.up_proj.weight"],
                    f"{prefix}.up_proj.weight"
                )
                block.ff.fc3[e].weight = assign(
                    block.ff.fc3[e].weight,
                    params[f"{prefix}.down_proj.weight"],
                    f"{prefix}.down_proj.weight"
                )
                # 分配权重后，将专家层从meta移动到CPU
                block.ff.fc1[e] = block.ff.fc1[e].to("cpu")
                block.ff.fc2[e] = block.ff.fc2[e].to("cpu")
                block.ff.fc3[e] = block.ff.fc3[e].to("cpu")

        else:
            block.ff.fc1.weight = assign(
                block.ff.fc1.weight,
                params[f"model.layers.{l}.mlp.gate_proj.weight"],
                f"model.layers.{l}.mlp.gate_proj.weight"
            )
            block.ff.fc2.weight = assign(
                block.ff.fc2.weight,
                params[f"model.layers.{l}.mlp.up_proj.weight"],
                f"model.layers.{l}.mlp.up_proj.weight"
            )
            block.ff.fc3.weight = assign(
                block.ff.fc3.weight,
                params[f"model.layers.{l}.mlp.down_proj.weight"],
                f"model.layers.{l}.mlp.down_proj.weight"
            )

        block.norm2.scale = assign(
            block.norm2.scale,
            params[f"model.layers.{l}.post_attention_layernorm.weight"],
            f"model.layers.{l}.post_attention_layernorm.weight"
        )

    # Final normalization and output head
    model.final_norm.scale = assign(model.final_norm.scale, params["model.norm.weight"], "model.norm.weight")

    if "lm_head.weight" in params:
        model.out_head.weight = assign(model.out_head.weight, params["lm_head.weight"], "lm_head.weight")
    else:
        # 模型使用权重绑定，因此我们在这里重用嵌入层权重
        print("Model uses weight tying.")
        model.out_head.weight = assign(model.out_head.weight, params["model.embed_tokens.weight"], "model.embed_tokens.weight")
```

2.4 整体代码

- 完整的代码实现可以在 `2_qwen3_archtecture_moe.py` 文件中找到，包括模型定义、权重加载、文本生成等功能。
- 使用示例

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-
#
# @File    : 2_qwen3_archtecture_moe.py
# @Date    : 2025/8/5
# @Author  : mengzhichao
# @Version : 1.0
# @Desc    :
import torch
import torch.nn as nn
import json
import os
from pathlib import Path
from safetensors.torch import load_file
from huggingface_hub import snapshot_download
import re
from tokenizers import Tokenizer
from pydantic import BaseModel, Field
from tqdm import tqdm
import time


class Qwen3MoeConfig(BaseModel):
    """Pydantic model for Qwen3 moe configuration"""
    model_config = {"arbitrary_types_allowed": True}

    vocab_size: int = Field(..., description="词汇表大小")
    context_length: int = Field(..., description="用于训练模型的上下文长度")
    emb_dim: int = Field(..., description="Embedding 维度")
    n_heads: int = Field(..., description="注意力头的数量")
    n_layers: int = Field(..., description="层数")
    head_dim: int = Field(..., description="GQA 中每个注意力头的维度大小")
    qk_norm: bool = Field(..., description="是否在 GQA 中对key和value进行归一化")
    n_kv_groups: int = Field(..., description="用于分组查询注意力的 KV 组数")
    rope_base: float = Field(..., description="RoPE 中'theta'的基数值")
    dtype: torch.dtype = Field(..., description="较低精度的数据类型，用于降低显存占用")
    # MoE 相关参数
    num_experts: int = Field(..., description="专家数量")
```

```python
    num_experts_per_tok: int = Field(..., description="每个token使用的专家数量")
    moe_intermediate_size: int = Field(..., description="MoE中间层大小")


class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.fc1 = nn.Linear(cfg.emb_dim, cfg.hidden_dim, dtype=cfg.dtype, bias=False)
        self.fc2 = nn.Linear(cfg.emb_dim, cfg.hidden_dim, dtype=cfg.dtype, bias=False)
        self.fc3 = nn.Linear(cfg.hidden_dim, cfg.emb_dim, dtype=cfg.dtype, bias=False)

    def forward(self, x):
        x_fc1 = self.fc1(x)
        x_fc2 = self.fc2(x)
        x = nn.functional.silu(x_fc1) * x_fc2
        return self.fc3(x)


class MoEFeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.num_experts_per_tok = cfg.num_experts_per_tok
        self.num_experts = cfg.num_experts
        self.gate = nn.Linear(cfg.emb_dim, cfg.num_experts, bias=False, dtype=cfg.dtype)

        # meta device to reduce memory pressure when initializing the model before loading weights
        meta_device = torch.device("meta")
        self.fc1 = nn.ModuleList([
            nn.Linear(
                cfg.emb_dim, cfg.moe_intermediate_size,
                bias=False, dtype=cfg.dtype, device=meta_device)
            for _ in range(cfg.num_experts)]
        )
        self.fc2 = nn.ModuleList([
            nn.Linear(
                cfg.emb_dim, cfg.moe_intermediate_size,
                bias=False, dtype=cfg.dtype, device=meta_device
            )
            for _ in range(cfg.num_experts)]
        )
        self.fc3 = nn.ModuleList([
            nn.Linear(
                cfg.moe_intermediate_size, cfg.emb_dim,
                bias=False, dtype=cfg.dtype, device=meta_device
            )
```

```python
            for _ in range(cfg.num_experts)]
        )

    def forward(self, x):
        b, seq_len, embed_dim = x.shape
        scores = self.gate(x)  # (b, seq_len, num_experts)
        topk_scores, topk_indices = torch.topk(scores, self.num_experts_per_tok, dim=-1)
        topk_probs = torch.softmax(topk_scores, dim=-1)

        expert_outputs = []
        for e in range(self.num_experts):
            hidden = torch.nn.functional.silu(self.fc1[e](x)) * self.fc2[e](x)
            out = self.fc3[e](hidden)
            expert_outputs.append(out.unsqueeze(-2))
        expert_outputs = torch.cat(expert_outputs, dim=-2)  # (b, t, num_experts, emb_dim)

        gating_probs = torch.zeros_like(scores)

        for i in range(self.num_experts_per_tok):
            indices = topk_indices[..., i:i + 1]
            prob = topk_probs[..., i:i + 1]
            gating_probs.scatter_(dim=-1, index=indices, src=prob)
        gating_probs = gating_probs.unsqueeze(-1)  # (b, t, num_experts, 1)

        # Weighted sum over experts
        y = (gating_probs * expert_outputs).sum(dim=-2)
        return y


class RMSNorm(nn.Module):
    def __init__(self, emb_dim, eps=1e-6, bias=False, qwen3_compatible=True):
        super().__init__()
        self.eps = eps
        self.qwen3_compatible = qwen3_compatible
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim)) if bias else None

    def forward(self, x):
        input_dtype = x.dtype

        if self.qwen3_compatible:
            x = x.to(torch.float32)

        variance = x.pow(2).mean(dim=-1, keepdim=True)
        norm_x = x * torch.rsqrt(variance + self.eps)
        norm_x = norm_x * self.scale
```

```python

        if self.shift is not None:
            norm_x = norm_x + self.shift

        return norm_x.to(input_dtype)


def compute_rope_params(head_dim, theta_base=10_000, context_length=4096, dtype=torch.float32):
    assert head_dim % 2 == 0, "Embedding dimension must be even"

    # Compute the inverse frequencies
    inv_freq = 1.0 / (theta_base ** (torch.arange(0, head_dim, 2, dtype=dtype)[: (head_dim // 2)].float() / head_dim))

    # Generate position indices
    positions = torch.arange(context_length, dtype=dtype)

    # Compute the angles
    angles = positions[:, None] * inv_freq[None, :]  # Shape: (context_length, head_dim // 2)

    # Expand angles to match the head_dim
    angles = torch.cat([angles, angles], dim=1)  # Shape: (context_length, head_dim)

    # Precompute sine and cosine
    cos = torch.cos(angles)
    sin = torch.sin(angles)

    return cos, sin


def apply_rope(x, cos, sin):
    # x: (batch_size, num_heads, seq_len, head_dim)
    batch_size, num_heads, seq_len, head_dim = x.shape
    assert head_dim % 2 == 0, "Head dimension must be even"

    # Split x into first half and second half
    x1 = x[..., : head_dim // 2]  # First half
    x2 = x[..., head_dim // 2:]  # Second half

    # Adjust sin and cos shapes
    cos = cos[:seq_len, :].unsqueeze(0).unsqueeze(0)  # Shape: (1, 1, seq_len, head_dim)
    sin = sin[:seq_len, :].unsqueeze(0).unsqueeze(0)

    # Apply the rotary transformation
    rotated = torch.cat((-x2, x1), dim=-1)
    x_rotated = (x * cos) + (rotated * sin)

```

```python
    # It's ok to use lower-precision after applying cos and sin rotation
    return x_rotated.to(dtype=x.dtype)


class GroupedQueryAttention(nn.Module):
    def __init__(
            self, d_in, num_heads, num_kv_groups, head_dim=None, qk_norm=False, dtype=None
    ):
        super().__init__()
        assert num_heads % num_kv_groups == 0, "num_heads must be divisible by num_kv_groups"

        self.num_heads = num_heads
        self.num_kv_groups = num_kv_groups
        self.group_size = num_heads // num_kv_groups

        if head_dim is None:
            assert d_in % num_heads == 0, "`d_in` must be divisible by `num_heads` if `head_dim` is not set"
            head_dim = d_in // num_heads

        self.head_dim = head_dim
        self.d_out = num_heads * head_dim

        self.W_query = nn.Linear(d_in, self.d_out, bias=False, dtype=dtype)
        self.W_key = nn.Linear(d_in, num_kv_groups * head_dim, bias=False, dtype=dtype)
        self.W_value = nn.Linear(d_in, num_kv_groups * head_dim, bias=False, dtype=dtype)

        self.out_proj = nn.Linear(self.d_out, d_in, bias=False, dtype=dtype)

        if qk_norm:
            self.q_norm = RMSNorm(head_dim, eps=1e-6)
            self.k_norm = RMSNorm(head_dim, eps=1e-6)
        else:
            self.q_norm = self.k_norm = None

    def forward(self, x, mask, cos, sin):
        b, num_tokens, _ = x.shape

        # Apply projections
        queries = self.W_query(x)  # (b, num_tokens, num_heads * head_dim)
        keys = self.W_key(x)  # (b, num_tokens, num_kv_groups * head_dim)
        values = self.W_value(x)  # (b, num_tokens, num_kv_groups * head_dim)
```

```python

        # Reshape
        queries = queries.view(b, num_tokens, self.num_heads, self.head_dim).transpose(1, 2)
        keys = keys.view(b, num_tokens, self.num_kv_groups, self.head_dim).transpose(1, 2)
        values = values.view(b, num_tokens, self.num_kv_groups, self.head_dim).transpose(1, 2)

        # Optional normalization
        if self.q_norm:
            queries = self.q_norm(queries)
        if self.k_norm:
            keys = self.k_norm(keys)

        # Apply RoPE
        queries = apply_rope(queries, cos, sin)
        keys = apply_rope(keys, cos, sin)

        # Expand K and V to match number of heads
        keys = keys.repeat_interleave(self.group_size, dim=1)
        values = values.repeat_interleave(self.group_size, dim=1)

        # Attention
        attn_scores = queries @ keys.transpose(2, 3)
        attn_scores = attn_scores.masked_fill(mask, -torch.inf)
        attn_weights = torch.softmax(attn_scores / self.head_dim ** 0.5, dim=-1)

        context = (attn_weights @ values).transpose(1, 2).reshape(b, num_tokens, self.d_out)
        return self.out_proj(context)


class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.att = GroupedQueryAttention(
            d_in=cfg.emb_dim,
            num_heads=cfg.n_heads,
            head_dim=cfg.head_dim,
            num_kv_groups=cfg.n_kv_groups,
            qk_norm=cfg.qk_norm,
            dtype=cfg.dtype
        )
        if cfg.num_experts > 0:
            self.ff = MoEFeedForward(cfg)
        else:
            self.ff = FeedForward(cfg)
        self.norm1 = RMSNorm(cfg.emb_dim, eps=1e-6)
        self.norm2 = RMSNorm(cfg.emb_dim, eps=1e-6)

```

```python
    def forward(self, x, mask, cos, sin):
        # Shortcut connection for attention block
        shortcut = x
        x = self.norm1(x)
        x = self.att(x, mask, cos, sin)  # Shape [batch_size, num_tokens, emb_size]
        x = x + shortcut  # Add the original input back

        # Shortcut connection for feed-forward block
        shortcut = x
        x = self.norm2(x)
        x = self.ff(x)
        x = x + shortcut  # Add the original input back

        return x


class Qwen3MoeModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()

        # Main model parameters
        self.tok_emb = nn.Embedding(cfg.vocab_size, cfg.emb_dim, dtype=cfg.dtype)

        self.trf_blocks = nn.ModuleList(
            # ModuleList since Sequential can only accept one input, and we need `x, mask, cos, sin`
            [TransformerBlock(cfg) for _ in range(cfg.n_layers)]
        )

        self.final_norm = RMSNorm(cfg.emb_dim)
        self.out_head = nn.Linear(cfg.emb_dim, cfg.vocab_size, bias=False, dtype=cfg.dtype)

        # Reusuable utilities
        if cfg.head_dim is None:
            head_dim = cfg.emb_dim // cfg.n_heads
        else:
            head_dim = cfg.head_dim
        cos, sin = compute_rope_params(
            head_dim=head_dim,
            theta_base=cfg.rope_base,
            context_length=cfg.context_length
        )
        self.register_buffer("cos", cos, persistent=False)
        self.register_buffer("sin", sin, persistent=False)
        self.cfg = cfg

    def forward(self, in_idx):
        # Forward pass
        tok_embeds = self.tok_emb(in_idx)
```

```python
        x = tok_embeds

        num_tokens = x.shape[1]
        mask = torch.triu(torch.ones(num_tokens, num_tokens, device=x.device, dtype=torch.bool), diagonal=1)

        for block in self.trf_blocks:
            x = block(x, mask, self.cos, self.sin)
        x = self.final_norm(x)
        logits = self.out_head(x.to(self.cfg.dtype))
        return logits


def model_memory_size(model, input_dtype=torch.float32):
    total_params = 0
    total_grads = 0
    for param in model.parameters():
        # Calculate total number of elements per parameter
        param_size = param.numel()
        total_params += param_size
        # Check if gradients are stored for this parameter
        if param.requires_grad:
            total_grads += param_size

    # Calculate buffer size (non-parameters that require memory)
    total_buffers = sum(buf.numel() for buf in model.buffers())

    # Size in bytes = (Number of elements) * (Size of each element in bytes)
    # We assume parameters and gradients are stored in the same type as input dtype
    element_size = torch.tensor(0, dtype=input_dtype).element_size()
    total_memory_bytes = (total_params + total_grads + total_buffers) * element_size

    # Convert bytes to gigabytes
    total_memory_gb = total_memory_bytes / (1024 ** 3)

    return total_memory_gb


def load_weights_into_qwen(model, param_config, params):
    def assign(left, right, tensor_name="unknown"):
        if left.shape != right.shape:
            raise ValueError(f"Shape mismatch in tensor '{tensor_name}'. Left: {left.shape}, Right: {right.shape}")
        return torch.nn.Parameter(right.clone().detach() if isinstance(right, torch.Tensor) else torch.tensor(right))

    model.tok_emb.weight = assign(model.tok_emb.weight, params["model.embed_tokens.weight"],
```

```
                                    "model.embed_tokens.weight")

    bar = tqdm(range(param_config.n_layers))
    for l in bar:
        bar.set_description(f"Loading weights into layer {l+1}")
        block = model.trf_blocks[l]
        att = block.att

        # Q, K, V projections
        att.W_query.weight = assign(
            att.W_query.weight,
            params[f"model.layers.{l}.self_attn.q_proj.weight"],
            f"model.layers.{l}.self_attn.q_proj.weight"
        )
        att.W_key.weight = assign(
            att.W_key.weight,
            params[f"model.layers.{l}.self_attn.k_proj.weight"],
            f"model.layers.{l}.self_attn.k_proj.weight"
        )
        att.W_value.weight = assign(
            att.W_value.weight,
            params[f"model.layers.{l}.self_attn.v_proj.weight"],
            f"model.layers.{l}.self_attn.v_proj.weight"
        )

        # Output projection
        att.out_proj.weight = assign(
            att.out_proj.weight,
            params[f"model.layers.{l}.self_attn.o_proj.weight"],
            f"model.layers.{l}.self_attn.o_proj.weight"
        )

        # QK norms
        if hasattr(att, "q_norm") and att.q_norm is not None:
            att.q_norm.scale = assign(
                att.q_norm.scale,
                params[f"model.layers.{l}.self_attn.q_norm.weight"],
                f"model.layers.{l}.self_attn.q_norm.weight"
            )
        if hasattr(att, "k_norm") and att.k_norm is not None:
            att.k_norm.scale = assign(
                att.k_norm.scale,
                params[f"model.layers.{l}.self_attn.k_norm.weight"],
                f"model.layers.{l}.self_attn.k_norm.weight"
            )

        # Attention layernorm
        block.norm1.scale = assign(
            block.norm1.scale,
            params[f"model.layers.{l}.input_layernorm.weight"],
```

```
                f"model.layers.{l}.input_layernorm.weight"
            )

            # Feedforward weights
            if "num_experts" in param_config.model_dump():
                # Load router (gating) weights
                block.ff.gate.weight = assign(
                    block.ff.gate.weight,
                    params[f"model.layers.{l}.mlp.gate.weight"],
                    f"model.layers.{l}.mlp.gate.weight"
                )
                # Load expert weights
                for e in range(param_config.num_experts):
                    prefix = f"model.layers.{l}.mlp.experts.{e}"
                    block.ff.fc1[e].weight = assign(
                        block.ff.fc1[e].weight,
                        params[f"{prefix}.gate_proj.weight"],
                        f"{prefix}.gate_proj.weight"
                    )
                    block.ff.fc2[e].weight = assign(
                        block.ff.fc2[e].weight,
                        params[f"{prefix}.up_proj.weight"],
                        f"{prefix}.up_proj.weight"
                    )
                    block.ff.fc3[e].weight = assign(
                        block.ff.fc3[e].weight,
                        params[f"{prefix}.down_proj.weight"],
                        f"{prefix}.down_proj.weight"
                    )
                    # After assigning weights, move the expert layers from meta to CPU
                    block.ff.fc1[e] = block.ff.fc1[e].to("cpu")
                    block.ff.fc2[e] = block.ff.fc2[e].to("cpu")
                    block.ff.fc3[e] = block.ff.fc3[e].to("cpu")

            else:
                block.ff.fc1.weight = assign(
                    block.ff.fc1.weight,
                    params[f"model.layers.{l}.mlp.gate_proj.weight"],
                    f"model.layers.{l}.mlp.gate_proj.weight"
                )
                block.ff.fc2.weight = assign(
                    block.ff.fc2.weight,
                    params[f"model.layers.{l}.mlp.up_proj.weight"],
                    f"model.layers.{l}.mlp.up_proj.weight"
                )
                block.ff.fc3.weight = assign(
                    block.ff.fc3.weight,
                    params[f"model.layers.{l}.mlp.down_proj.weight"],
                    f"model.layers.{l}.mlp.down_proj.weight"
                )

```

```
        block.norm2.scale = assign(
            block.norm2.scale,
            params[f"model.layers.{l}.post_attention_layernorm.weight"],
            f"model.layers.{l}.post_attention_layernorm.weight"
        )

    # Final normalization and output head
    model.final_norm.scale = assign(model.final_norm.scale, params["model.norm.weight"], "model.norm.weight")

    if "lm_head.weight" in params:
        model.out_head.weight = assign(model.out_head.weight, params["lm_head.weight"], "lm_head.weight")
    else:
        # Model uses weight tying, hence we reuse the embedding layer weights here
        print("Model uses weight tying.")
        model.out_head.weight = assign(model.out_head.weight, params["model.embed_tokens.weight"],
"model.embed_tokens.weight")


class Qwen3Tokenizer:
    _SPECIALS = [
        "<|endoftext|>",
        "<|im_start|>", "<|im_end|>",
        "<|object_ref_start|>", "<|object_ref_end|>",
        "<|box_start|>", "<|box_end|>",
        "<|quad_start|>", "<|quad_end|>",
        "<|vision_start|>", "<|vision_end|>",
        "<|vision_pad|>", "<|image_pad|>", "<|video_pad|>",
    ]
    _SPLIT_RE = re.compile(r"(<\|[^>]+?\|>)")

    def __init__(self, tokenizer_file_path="tokenizer.json", repo_id=None,
                 apply_chat_template=True, add_generation_prompt=False, add_thinking=False):

        self.apply_chat_template = apply_chat_template
        self.add_generation_prompt = add_generation_prompt
        self.add_thinking = add_thinking

        tok_file = Path(tokenizer_file_path)
        self._tok = Tokenizer.from_file(str(tok_file))
        self._special_to_id = {t: self._tok.token_to_id(t) for t in self._SPECIALS}

        self.pad_token_id = self._special_to_id.get("<|endoftext|>")
```

```
        self.eos_token_id = self.pad_token_id

        if repo_id and "Base" not in repo_id:
            eos_token = "<|im_end|>"
        else:
            eos_token = "<|endoftext|>"
        if eos_token in self._special_to_id:
            self.eos_token_id = self._special_to_id[eos_token]

    def encode(self, text, chat_wrapped=None):
        if chat_wrapped is None:
            chat_wrapped = self.apply_chat_template

        stripped = text.strip()
        if stripped in self._special_to_id and "\n" not in stripped:
            return [self._special_to_id[stripped]]

        if chat_wrapped:
            text = self._wrap_chat(text)

        ids = []
        for part in filter(None, self._SPLIT_RE.split(text)):
            if part in self._special_to_id:
                ids.append(self._special_to_id[part])
            else:
                ids.extend(self._tok.encode(part).ids)
        return ids

    def decode(self, ids):
        return self._tok.decode(ids, skip_special_tokens=False)

    def _wrap_chat(self, user_msg):
        s = f"<|im_start|>user\n{user_msg}<|im_end|>\n"
        if self.add_generation_prompt:
            s += "<|im_start|>assistant"
            if self.add_thinking:
                s += "\n"
            else:
                s += "\n<think>\n\n</think>\n\n"
        return s


def generate_text_basic_stream(model, token_ids, max_new_tokens, eos_token_id=None):
    model.eval()
    with torch.no_grad():
        for _ in range(max_new_tokens):
            out = model(token_ids)[:, -1]
            next_token = torch.argmax(out, dim=-1, keepdim=True)

            if (eos_token_id is not None
```

```
                    and torch.all(next_token == eos_token_id)):
                break

            yield next_token

            token_ids = torch.cat([token_ids, next_token], dim=1)


def set_qwen3_moe_config(repo_id):
    repo_id_list = ["Qwen/Qwen3-30B-A3B", "Qwen/Qwen3-235B-A22B-Instruct-2507",
                    "Qwen/Qwen3-30B-A3B-Thinking-2507", "Qwen/Qwen3-Coder-30B-A3B-Instruct",
                    "/root/Qwen/Qwen3-30B-A3B-Instruct-2507"]
    assert repo_id in repo_id_list, f"repo_id  error,must be one of {repo_id_list}"

    list_30B = ["Qwen/Qwen3-30B-A3B", "Qwen/Qwen3-30B-A3B-Thinking-2507", "Qwen/Qwen3-Coder-30B-A3B-Instruct",
                "/root/Qwen/Qwen3-30B-A3B-Instruct-2507"]
    list_235B = ["Qwen/Qwen3-235B-A22B-Instruct-2507"]

    if repo_id in list_30B:
        # 30B
        QWEN3_CONFIG = {
            "vocab_size": 151_936,
            "context_length": 262_144,
            "emb_dim": 2048,
            "n_heads": 32,
            "n_layers": 48,
            "head_dim": 128,
            "qk_norm": True,
            "n_kv_groups": 4,
            "rope_base": 10_000_000.0,
            "dtype": torch.bfloat16,
            "num_experts": 128,
            "num_experts_per_tok": 8,
            "moe_intermediate_size": 768,
        }
    elif repo_id in list_235B:
        ## 235B
        QWEN3_CONFIG = {
            "vocab_size": 151_936,
            "context_length": 262_144,
            "emb_dim": 4096,
            "n_heads": 64,
            "n_layers": 94,
            "head_dim": 128,
            "qk_norm": True,
            "n_kv_groups": 4,
            "rope_base": 5_000_000.0,
            "dtype": torch.bfloat16,
```

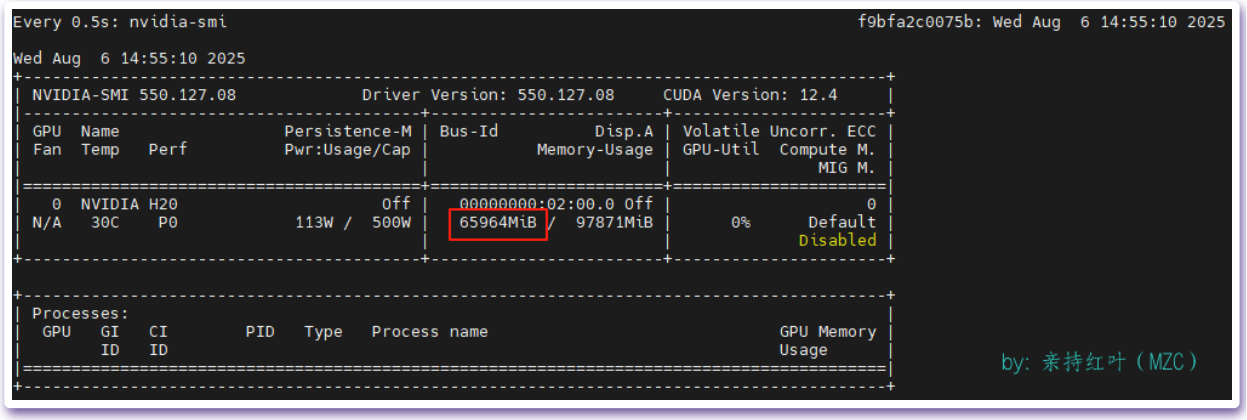
```
            "num_experts": 128,
            "num_experts_per_tok": 8,
            "moe_intermediate_size": 1536,
        }
    else:
        raise ValueError(f"{repo_id} is not supported.")

    return Qwen3MoeConfig(**QWEN3_CONFIG)


if __name__ == "__main__":

    # repo_id = "Qwen/Qwen3-30B-A3B"  # Original Instruct/Thinking hybrind model
    # repo_id = "Qwen/Qwen3-235B-A22B-Instruct-2507"  # New instruct model
    # repo_id = "Qwen/Qwen3-30B-A3B-Thinking-2507"  # New thinking model
    # repo_id = "Qwen/Qwen3-Coder-30B-A3B-Instruct"  # (Qwen3 Coder Flash)
    repo_id = "/root/Qwen/Qwen3-30B-A3B-Instruct-2507" # 模型提前下载在这个路径

    qwen3_moe_cfg = set_qwen3_moe_config(repo_id)

    if torch.cuda.is_available():
        device = torch.device("cuda")
    elif torch.backends.mps.is_available():
        device = torch.device("mps")
    else:
        device = torch.device("cpu")

    print(device)

    torch.manual_seed(123)

    with device:
        model = Qwen3MoeModel(qwen3_moe_cfg)

    # model.to(device)

    print(f"model: \n{model}")

    total_params = sum(p.numel() for p in model.parameters())
    print(f"Total number of parameters: {total_params:,}")

    # Account for weight tying
    total_params_normalized = total_params - model.tok_emb.weight.numel()
    print(f"\nTotal number of unique parameters: {total_params_normalized:,}")

```

```python
    print(f"float32 (PyTorch default): {model_memory_size(model, input_dtype=torch.float32):.2f} GB")
    print(f"bfloat16: {model_memory_size(model, input_dtype=torch.bfloat16):.2f} GB")

    # local_dir = Path(repo_id).parts[-1]

    # repo_dir = snapshot_download(repo_id=repo_id, local_dir=local_dir)
    repo_dir = "/root/Qwen/Qwen3-30B-A3B-Instruct-2507"
    index_path = os.path.join(repo_dir, "model.safetensors.index.json")
    with open(index_path, "r") as f:
        index = json.load(f)

    weights_dict = {}
    for filename in set(index["weight_map"].values()):
        shard_path = os.path.join(repo_dir, filename)
        shard = load_file(shard_path)
        weights_dict.update(shard)

    load_weights_into_qwen(model, qwen3_moe_cfg, weights_dict)
    model.to(device);

    tokenizer_file_path = f"{repo_dir}/tokenizer.json"

    tokenizer = Qwen3Tokenizer(
        tokenizer_file_path=tokenizer_file_path,
        repo_id=repo_id,
        add_generation_prompt=True,
        add_thinking=True
    )

    prompt = "使用python实现一个二分查找的函数"

    input_token_ids = tokenizer.encode(prompt)
    text = tokenizer.decode(input_token_ids)

    input_token_ids_tensor = torch.tensor(input_token_ids, device=device).unsqueeze(0)
    start_time = time.perf_counter()
    llm_response = ""
    token_num = 0
    for token in generate_text_basic_stream(
            model=model,
            token_ids=input_token_ids_tensor,
            max_new_tokens=500,
            # eos_token_id=tokenizer.eos_token_id
    ):
        token_id = token.squeeze(0).tolist()
        current_token = tokenizer.decode(token_id)
        llm_response += current_token
```

```python
            token_num += 1
            print(
                current_token,
                end="",
                flush=True
            )
        end_time = time.perf_counter()
        print(f"共生成token数: {token_num}, 耗时: {end_time - start_time}, tps: {token_num / (end_time - start_time)}")
        print(f"中文字符数: {len(current_token)}, 耗时: {end_time - start_time}, 每秒中文字符数: {len(current_token) / (end_time - start_time)}")


```

- 显存占用（H20上感觉只有3-4tokens/s）



- 输出示例

```
(py310) root@f9bfa2c0075b:~# python h20.py
cuda
model:
Qwen3MoeModel(
  (tok_emb): Embedding(151936, 2048)
  (trf_blocks): ModuleList(
    (0-47): 48 x TransformerBlock(
      (att): GroupedQueryAttention(
        (W_query): Linear(in_features=2048, out_features=4096, bias=False)
        (W_key): Linear(in_features=2048, out_features=512, bias=False)
        (W_value): Linear(in_features=2048, out_features=512, bias=False)
        (out_proj): Linear(in_features=4096, out_features=2048, bias=False)
        (q_norm): RMSNorm()
        (k_norm): RMSNorm()
      )
      (ff): MoEFeedForward(
        (gate): Linear(in_features=2048, out_features=128, bias=False)
        (fc1): ModuleList(
          (0-127): 128 x Linear(in_features=2048, out_features=768, bias=False)
        )
        (fc2): ModuleList(
          (0-127): 128 x Linear(in_features=2048, out_features=768, bias=False)
        )
        (fc3): ModuleList(
          (0-127): 128 x Linear(in_features=768, out_features=2048, bias=False)
        )
      )
      (norm1): RMSNorm()
      (norm2): RMSNorm()
    )
  )
  (final_norm): RMSNorm()
  (out_head): Linear(in_features=2048, out_features=151936, bias=False)
)
Total number of parameters: 30,532,122,624

Total number of unique parameters: 30,220,957,696
float32 (PyTorch default): 227.73 GB
bfloat16: 113.87 GB
下面是一个使用 Python 实现的**二分查找（Binary Search）**函数，适用于**已排序的列表**。

### 🔍 二分查找函数（递归实现）

```python
def binary_search_recursive(arr, target, left=0, right=None):
    """
    二分查找（递归实现）

    参数:
        arr: 🔢排序的列表
        target: 🎯查找的目标值
        left: 📍索范围的左边界（包含）
        right: 📍索范围的右边界（包含）

    返回:
        如果找到目标，返回其索引；否则返回 -1
    """
    if right is None:
        right = len(arr) - 1

    # 🚨础情况：搜索范围无效
    if left > right:
        return -1

    mid = (left + right) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, right)
    else:
        return binary_search_recursive(arr, target, left, mid - 1)
```

🔍 二分查找函数（迭代实现）
```

by: 亲持红叶（MZC）